

國立成功大學
電機工程研究所
碩士論文

使用任務發派管理及基於強化學習之動態電壓及
頻率調整之高能源效率通用計算圖形處理器

**An Energy-Efficient GPGPU using Task Management
and DVFS Based on Reinforcement Learning**

研究生：蔡宗蕙

Student: Zong-Hui Tsai

指導教授：邱瀝毅 博士

Advisor: Lih-Yih Chiou, Ph.D.

**Department of Electrical Engineering
National Cheng Kung University,
Tainan, Taiwan, R.O.C
Thesis for Master of Science**

January, 2021

國立成功大學

碩士論文

使用任務發派管理及基於強化學習之動態電壓及頻率調整之高能源
效率通用計算圖形處理器

An Energy-Efficient GPGPU using Task Management and
DVFS Based on Reinforcement Learning

研究生：蔡宗蕙

本論文業經審查及口試合格特此證明

論文考試委員：

陳中弘
鄭致宏
邱瀝毅

指導教授：邱瀝毅

系(所)主管：林志隆

中華民國 110 年 1 月 4 日

An Energy-Efficient GPGPU using Task Management and DVFS Based on Reinforcement Learning

By

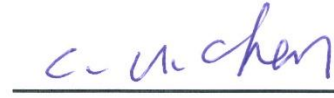
Zong-Hui Tsai

A thesis submitted to the graduate division in partial
fulfillment of the requirement for the degree of
Master of Science

at

National Cheng Kung University
Tainan, Taiwan, Republic of China
January, 2021

Approved by :

 _____

Advisor :



Chairman :



使用任務發派管理及基於強化學習之動態電壓及 頻率調整之高能源效率通用計算圖形處理器

研 究 生：蔡宗蕙 指導教授：邱瀝毅 博士

國立成功大學電機工程研究所 VLSI/CAD 組

摘 要

隨著終端運算市場蓬勃發展，圖型處理器(GPU)高功耗和高能量消耗的帶來的問題也逐漸變得不可忽視，起因於這些裝置無額外冷卻機制且使用壽命及可靠度取決於電池容量。然電池容量受到材料化學特性的限制下發展緩慢，提升能量的使用效率成為更實際的解法。

我們使用 double deep Q-learning 動態調整 GPGPU 的電壓和頻率降低能量消耗，並調控各個串流處理器(streaming processor; SM)的工作數量緩解快取記憶體資源搶佔的問題以提升性能。我們的方法不僅可以提升 18% 的性能，還可以省下 59% 的能量消耗。此外無論兩種極端特性的程式如何排列，皆可以低於 5% 的性能損失節省 53% 的能量消耗。

關鍵字：圖形處理器功率管理，動態電壓頻率調整，圖形處理器工作分配，強化學習。

An Energy-Efficient GPGPU using Task Management and DVFS Based on Reinforcement Learning

Student: Zong-Hui Tsai Advisor: Lih-Yih Chiou, Ph.D.

Department of Electrical Engineering
National Cheng Kung University,
Tainan, Taiwan, R.O.C

Abstract

GPGPU power and energy issues arise with expansion of edge-computing markets, because there are no embedded cooling devices in these devices and their lifetime and reliability is limited by battery capacity. Since capacity is restricted by chemical characteristics of battery materials, increasing energy-efficiency is a much more practical solution.

We propose a dynamic voltage and frequency scaling mechanism based on reinforcement learning at GPGPU runtime for energy-saving. Furthermore, we reduce cache contention by tuning the number of tasks in every SM to boost performance. With various benchmarks, the mechanism achieves 59% energy savings with 18% performance improvement on average. Moreover, the model shows strong adaptability when running programs interleaved by compute-intensive and memory-intensive benchmarks. The energy-saving is more than 53% with less than 5% performance loss for all sequences under experiments.

Key words: GPGPU power management, dynamic voltage and frequency scaling, task dispatch, reinforcement learning

誌 謝

首先感謝邱瀝毅老師，和老師報告研究進度時，老師總會引導我們思考實驗數據背後的真正原因及意義，並且教導我們如何以更引人注目的方式呈現自己的研究成果。也感謝老師讓我能在研究生涯中參加實習及各種研討會，開拓自己的眼界與人脈。而研究生涯中也有不少擔任助教的機會，當助教除了撰寫書面的作業要求，還需上台向學生簡述作業目標及驗證機制，從學生的問題，就可以得知自己那些地方的思慮不夠周全、哪張投影片內容表達得不夠清楚，讓我體會到「教學相長」此句話的意思。

再來，感謝學長維軒、俊興、柏丞、哲彬、健棠、晉維、峻豪、憲譚、昱宏、育緯、紀寬、唯宏在我遇到困難時，總不吝於給予建議及幫助，讓我即使在研究低潮，依然慶幸自己沒有選錯實驗室。感謝同學志展、Tenzing、錫峰、聖凱、偉碩，有你們一起在研究的路上互相提拔成長，是我莫大的榮幸。感謝學弟妹韻如、中凱、琬筠、心妤、晨哲、鎮中、子毅、泓名、芝儀、宗琪、宗翰、士瑩、昶融總給實驗室帶來歡樂的氣氛。

最後感謝家人經濟上和精神上的支持，使我得以挺過這條比學士艱辛數十倍的路程。

西元 2021 年 1 月於台南 蔡宗蕙 謹誌

Contents

摘要	i
Abstract.....	ii
Contents.....	iv
List of Tables	vi
List of Figures.....	vii
CHAPTER 1 INTRODUCTION	1
1.1 OVERVIEW	1
1.2 MOTIVATION	5
1.3 CONTRIBUTIONS	6
1.4 THESIS ORGANIZATION.....	6
CHAPTER 2 BACKGROUND.....	7
2.1 GENERAL-PURPOSE GRAPH PROCESSING UNIT	7
2.2 DOUBLE DEEP Q-LEARNING	8
CHAPTER 3 RELATED WORKS	12
3.1 DEFINITION OF COMPUTE-INTENSIVE, MEMORY-INTENSIVE AND HYBRID [13]	12
3.2 SHIFTING OF COMPUTE-BOUND AND MEMORY-BOUND [21].....	13
3.3 CONTENTION REDUCTION [16].....	15
3.4 DDQN ON GPU POWER MANAGEMENT [25]	18
3.5 SUMMARY OF RELATED WORKS	20
CHAPTER 4 THE PROPOSED GPU POWER MANAGEMENT	23
4.1 CACHE-CONTENTION-AWARE TASK DISPATCH ALGORITHM	23
4.2 DVFS BASED ON DDQN	30

4.2.1 GPU Features.....	31
CHAPTER 5 EXPERIMENTAL RESULTS AND ANALYSIS	34
5.1 ENVIRONMENT SETUP.....	34
5.1.1 GPU Virtual Platform Power Model	35
5.2 EXPERIMENTAL RESULTS.....	36
5.2.1 Experiment 1	36
5.2.2 Experiment 2	37
5.2.3 Experiment 3	39
5.2.4 Experiment 4	44
5.3 OVERHEAD	47
CHAPTER 6 CONCLUSIONS AND FUTURE WORKS.....	50
6.1 CONCLUSIONS	50
6.2 FUTURE WORKS	50
Reference.....	51

List of Tables

Table 2-1.	Action and reward decision	11
Table 3-1.	Frequency scaling based on different objectives	18
Table 3-2.	Related works summary	22
Table 5-1.	Platform setting	34
Table 5-2.	DDQN hyperparameter.....	34
Table 5-3.	Detailed overhead of every component	48



List of Figures

Figure 1-1.	Edge-computing market size prediction [6]	2
Figure 1-2.	Concept of [7] and [8]	2
Figure 1-3.	Power and performance prediction model of [19, 20].....	5
Figure 2-1.	GPU architecture	7
Figure 2-2.	SM architecture.....	8
Figure 2-3.	Reinforcement learning	9
Figure 2-4.	Q-table	9
Figure 2-5.	Q-Network [23]	10
Figure 2-6.	Training of Q-Network [24]	11
Figure 3-1.	Examples of DVFS impact on performance [21]	14
Figure 3-2.	DVFS impact on performance classification [21]	15
Figure 3-3.	Equalizer overview [16]	16
Figure 3-4.	Interaction between DDQN agent and GPU [25].....	19
Figure 3-5.	Timeline of feature collections and prediction [25]	20
Figure 4-1.	Number of memory-related task in every SM.....	24
Figure 4-2.	The severity of cache contention when executing different benchmarks.....	24
Figure 4-3.	Several SMs with cache contention problem	25
Figure 4-4.	More severe contention in SM 2	25
Figure 4-5.	CCATD (SM side)	27
Figure 4-6.	CCATD (Task Dispatcher side is modified from (a) to (b))	28
Figure 4-7.	Interaction between SM and Task Dispatcher	28
Figure 4-8.	SMs with cache contention request less tasks	29
Figure 4-9.	Under-utilized SMs request more tasks.....	30

Figure 4-10. System architecture.....	31
Figure 4-11. Blocks with feature monitors monitor (marked by green color)	33
Figure 5-1. Flow for power model [29].....	35
Figure 5-2. Error rate of power model [29].....	36
Figure 5-3. Average energy in different β settings	37
Figure 5-4. Average performance in different β settings.....	37
Figure 5-5. Timeline in experiment 2 and experiment 3	38
Figure 5-6. Energy comparison in different benchmarks	39
Figure 5-7. Performance comparison in different benchmarks	39
Figure 5-8. R_{ins} and R_{mem} of all benchmarks	40
Figure 5-9. Normalized energy in every benchmark (1st execution).....	42
Figure 5-10. Normalized energy in every benchmark (2nd execution)	42
Figure 5-11. Normalized energy in every benchmark (10th execution)	42
Figure 5-12. Normalized energy in every series (10th execution)	42
Figure 5-13. Normalized performance in every benchmark	43
Figure 5-14. Normalized execution time in every series (10th execution)	43
Figure 5-15. Sytem Architecture in Experiment 4	44
Figure 5-16. Action-decision on RISC-V CPU	45
Figure 5-17. Timeline of action-decision and learning flow	46
Figure 5-18. Energy when inference on RISC-V CPU	47
Figure 5-19. Performance when inference on RISC-V CPU	47
Figure 5-20. Energy overhead	48
Figure 5-21. Time overhead	49

Chapter 1 Introduction

In section 1.1, we overview GPU power and energy issues and several management methods have been proposed. Then, we discuss motivation of the work and our contributions in section 1.2 and 1.3, respectively. In section 1.4, we present the thesis organization.

1.1 Overview

The general-purpose graphics processing units (GPGPUs) play a crucial role in high performance clusters currently. They significantly boost performance of numerous applications in commercial and scientific fields, such as machine learning [1-3] and autonomous vehicles [4].

Edge-computing markets like autonomous vehicles, mobile phone, tablets grow rapidly since 2016 as illustrated in Figure 1-1. However, these edge devices face many more severe power and energy issues than cloud devices since they are not embedded with cooling devices. Their lifetime and reliability are restricted by battery capacity. Generally speaking, two solutions are used to extend the battery lifetime: one is to expand battery capacity and the other is to increase energy-efficiency. Nevertheless, the battery capacity grows only 5% per year because it is limited by chemical characteristics of materials [5]. Hence, increasing energy-efficiency is much more practical to solve these issues. GPGPUs, which dominated power consumption of these edge devices, is the component we focus on in this thesis.

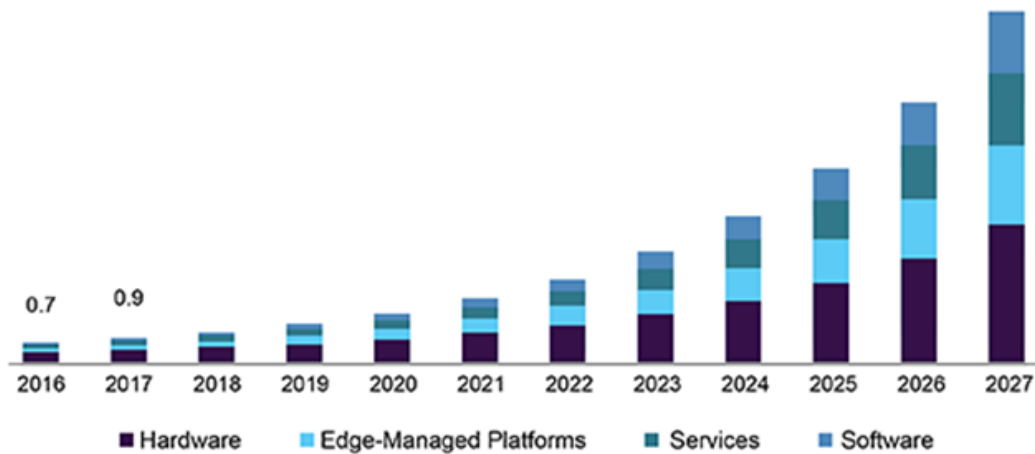


Figure 1-1. Edge-computing market size prediction [6]

GPUs power management methods can be classified into four categories as below.

1. Architecture-level redesign

Gebhart et al. inserted a cache between the register file and the arithmetic logic unit (ALU) to reduce accesses to the register file and to shorten data path [7]. Furthermore, they partitioned memory into a register file, a cache, and a shared memory based on kernel requirements [8]. We illustrate their key concepts in Figure 1-2.

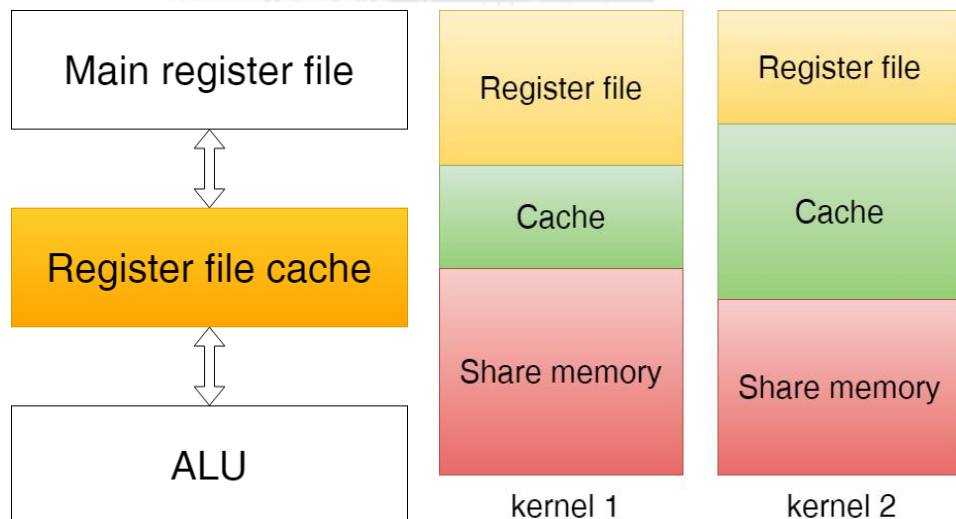


Figure 1-2. Concept of [7] and [8]

2. Dynamic resource allocation

Hong et al. predicted the optimal number of active processors which would achieve the highest performance per watt for given applications [9].

Wang et al. decided the number of active cores depending on code profile [10].

3. Programming-level adjustment

Wang et al. fused two kernels into a single thread for higher hardware utilization [11]. Yang et al. adjusted thread block dimension in GPU projects, like computation physics, machine learning, mathematics, to improve reusing rate of global memory data for higher energy-efficiency [12].

4. Dynamic voltage and frequency scaling

Dynamic voltage and frequency scaling (DVFS) has been proved as an effective approach to reduce GPU power consumption [13-22]. Despite the maturity of CPU DVFS, the GPU DVFS study just started around ten years ago.

Mei et al. conducted experiments on a real GPU platform with 37 benchmark applications and showed that voltage/frequency scaling is an effective approach to saving energy [15]. They found that the generally believed concept for CPUs, “high processor frequency would reduce energy because of shorter execution time”, is not always true for CPUs, especially on memory-intensive benchmark.

As scaling GPU core and memory frequency/voltage based on the kernel characteristics shows significant impact on energy-saving. Several mechanisms have been proposed to address power issues of GPU.

Sethia et al. observed that most GPU applications are bottlenecked by the number of compute resources, available memory bandwidth or limited size of data caches [16]. Aside from DVFS, they adjusted the number of concurrent

threads to reduce cache contention for performance-improvement and energy-saving.

Since GPUs are widely adopted in various fields nowadays, it is challenging to figure out a single rule which fits all applications and users' behaviors. To solve the problem, machine learning is involved in power management techniques to select the optimal hardware frequency/voltage setting. We divide these mechanisms into two subfields, supervised learning and unsupervised learning.

1. Supervised learning

Junke et al. indicated the relation between appropriate frequency and runtime characteristics are nonlinear; therefore, they used GA-BP neural network to represent the nonlinear function [18]. Jiao et al. executed 215 kernels at all 36 frequency settings to find out the most energy-efficient frequency setting for each kernel [17]. They trained neural network which takes kernel characteristics as inputs and outputs optimal frequency setting. Majumdar et al. and Fan et al. adopted a neural network to predict performance and power with hardware configuration and kernel features to optimize energy-efficiency and pareto set, respectively [19, 20]. We illustrate the model in Figure 1-3. João et al. stated that a kernel which is classified as compute-intensive may become memory-intensive when scaling up the core frequency or scaling down the memory frequency [21]. He sorted kernels into four categories depending on their sensitivity to core/memory frequency change and recorded their characteristics at default setting time. He collected these data to train a neural network which takes kernel characteristics as input and outputs the category.

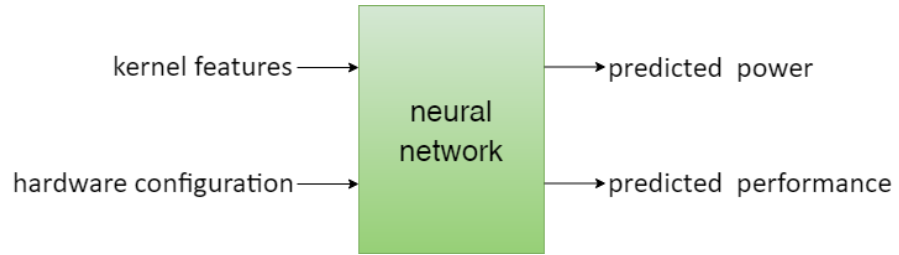


Figure 1-3. Power and performance prediction model of [19, 20]

2. Unsupervised learning

Ma et al. assumed suitable frequency and utilizations are linearly mapped [14]. They adopted weighted majority algorithm (WMA) to select the core-memory frequency pair after calculating loss function for weight updating. Somdip et al. aimed to optimize Quality of Service (QoS) on mobile device [22]. They chose Q-learning to optimize power consumption and peak thermal depending on users' behavior.

1.2 Motivation

The problem of GPU power and energy arises with expansion of edge-computing markets and DVFS has been proved as an effective approach for GPU energy-saving.

Nonetheless, the rule-based power management mentioned in section 1.1 sets threshold based on single GPU architecture, it is doubtful to apply on other GPU platforms. Furthermore, the effectiveness of the design was verified with less than 20 benchmarks from two suites. Supervised learning was proposed to solve this issue. Nevertheless, to assure that their mechanisms were effective on all applications, these authors made efforts to collect or create kernels contained all possible characteristics. Moreover, some of the works above noted that the most time-consuming task is not training the neural network but executing all kernels at all available hardware configurations [17, 19, 20]. As the authors depicted, they need to execute GPU 7k times,

4k times, 38k times, respectively. Authors in [14] and [22] adopted unsupervised learning that collects training set at runtime; therefore, they did not have to run all kernels at all possible hardware settings. Nonetheless, the assumption in [14], the relationship between utilization and suitable frequency, is only validated on CPU, but not GPU.

To implement GPU power management which dynamically fits any type of applications or users' behavior patterns, we propose an unsupervised-learning based mechanism that can release cache contention at runtime. Our proposed methodology achieves energy-saving and do not need to collect kernels from all applications and execute them at all hardware configurations beforehand.

1.3 Contributions

We propose to use the dynamic voltage and frequency scaling mechanism based on unsupervised learning at GPU runtime for energy-saving and to reduce component idle conditions caused by cache contention to boost performance.

1.4 Thesis Organization

The rest of the thesis is organized as follows. In Chapter 2, we present the background of our algorithm, GPU architecture and Double Deep Q-learning (DDQN). Chapter 3 discusses works related to our methodology in detail. Chapter 4 presents the proposed methodology. Chapter 5 shows the experiment setup and compares the proposed methodologies with prior works. Finally, Chapter 6 draws conclusions and describes future work.

Chapter 2 Background

For a better understanding of our work, we cover basic knowledge of GPU and DDQN in section 2.1 and 2.2, respectively.

2.1 General-Purpose Graph Processing Unit

A typical GPGPU consists of a task dispatcher, interconnection network, memory partitions and several SMs (streaming multiprocessors). A SM can be divided into front-end and back-end as depicted in Figure 2-1. In front-end pipeline, a SM receives instruction warps from Task Dispatcher. Instructions in the same warp have the same operator but different operands. One of these warps will be selected for SIMD (single instruction multiple data) execution in the back-end pipeline.

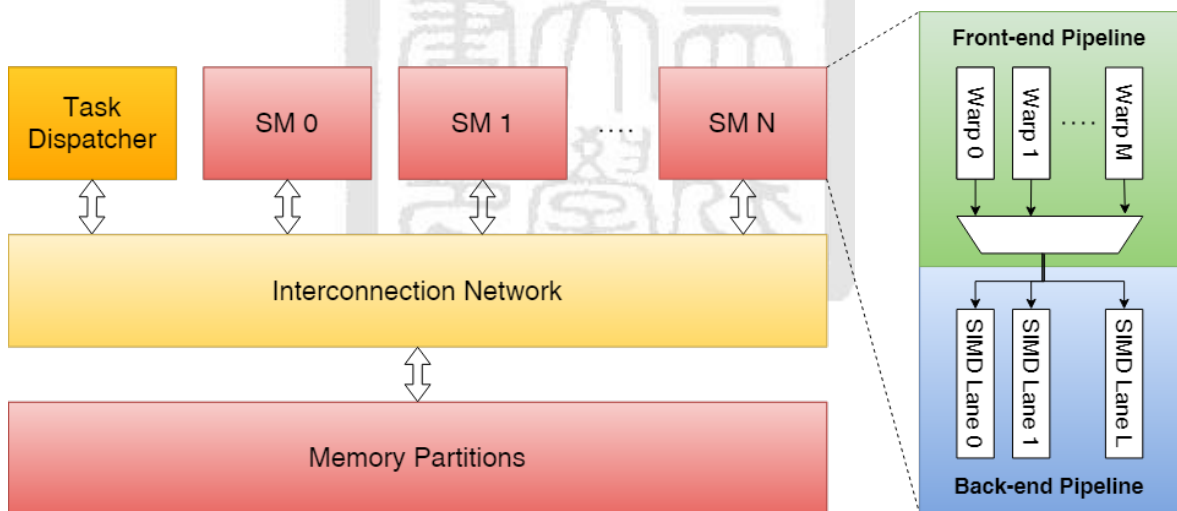


Figure 2-1. GPU architecture

We show the adopted SM architecture as shown in Figure 2-2. A warp is responsible for instruction fetch if there are no valid instructions in Instruction Buffer, and all warps in Instruction Buffer access I-Cache in round robin order. A warp's status can be represented with valid bit and ready bit. Once a warp is scheduled for an instruction fetch, its valid bit is not set until all instruction in this warp are issued to the backend-pipeline.

By contrast, the ready bit is set after assuring no data hazard in Dependency Check Unit. Some of Operand Collector Unit are allocated to a warp for operands from Register Bank. Warp Scheduler selects an instruction warp to execute in the backend-pipeline that is valid, ready and collected all operands. The compute warp will be executed in Execution Unit while the memory warp will be processed in Load/Store Unit. The memory warp will be processed in Load/Store Unit.

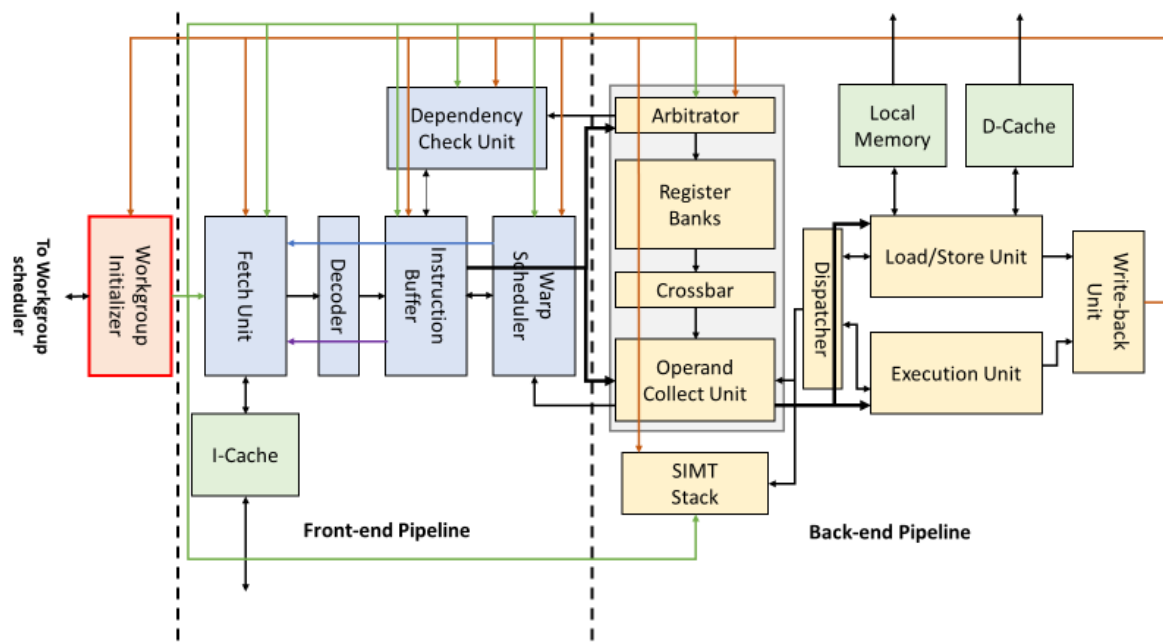


Figure 2-2. SM architecture

2.2 Double Deep Q-learning

Q-learning is an implementation of reinforcement learning, a subset of the unsupervised learning. As shown in Figure 2-3, there are two characters in reinforcement learning, agent and environment. The agent decides an action that the environment should take according to the current state. After the environment executes the action, it gives a reward to the agent to modify action-decision policy.

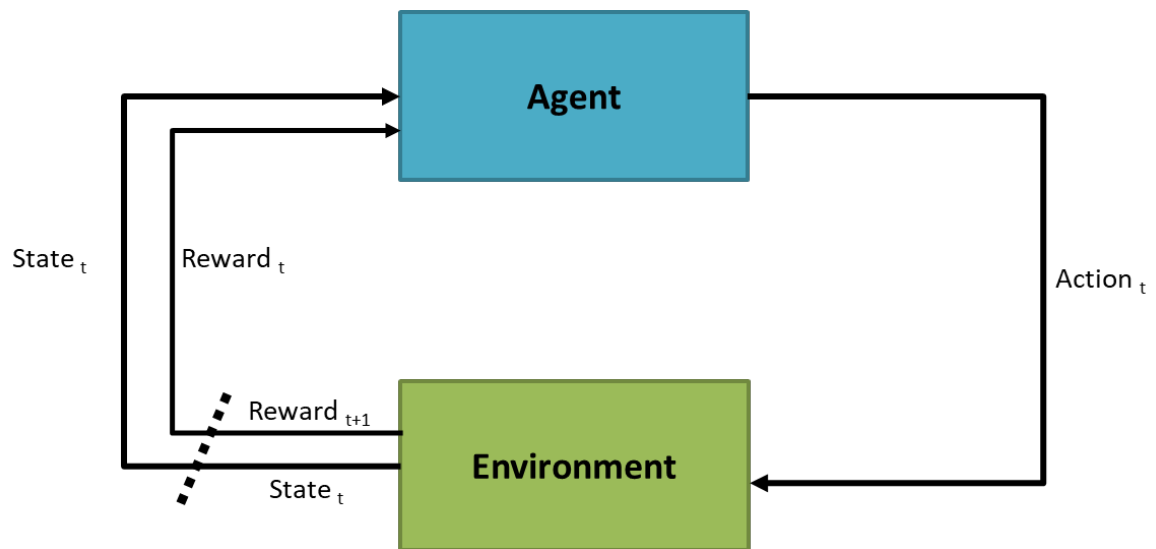


Figure 2-3. Reinforcement learning

In Q-learning, the agent refers as Q-table, which records rewards of different state/action pairs, to decide an action. It will take the action maximizing Q-value, which is the total rewards obtained between the first state and the last state. As Figure 2-4 depicts, it prefers to execute action 2 on state 1, and action 1 on state 4.

	Action 1	Action 2
State 1	0	5
State 2	0	5
State 3	0	5
State 4	20	0

Figure 2-4. Q-table

The agent's strategy can be expressed as Equation (2-1) ($[s, a, r, s']$ =[current state, action, reward, next state]). Because $Q(s', a)$ again depends on $Q(s'', a)$, we can expand $Q(s, a)$ as shown in Equation (2-2). γ reduces the contribution of rewards further in the future.

$$Q(s, a) = r(s, a) + \gamma \max_{a'} (Q(s', a')) \quad (2-1)$$

$$Q(s, a) = r(s, a) + \gamma r(s', a') + \gamma^2 r(s'', a'') + \dots + \gamma^n r(s^n, a^n) \quad (2-2)$$

Nonetheless, as the number of states and actions grows rapidly, it takes a large amount of memory space to record all state/action pairs. Therefore, one replaces the Q-table with network, which is called Q-network as presented in Figure 2-5. This algorithm is named Deep Q-learning (DQN). The agent takes an action with the most Q-value.

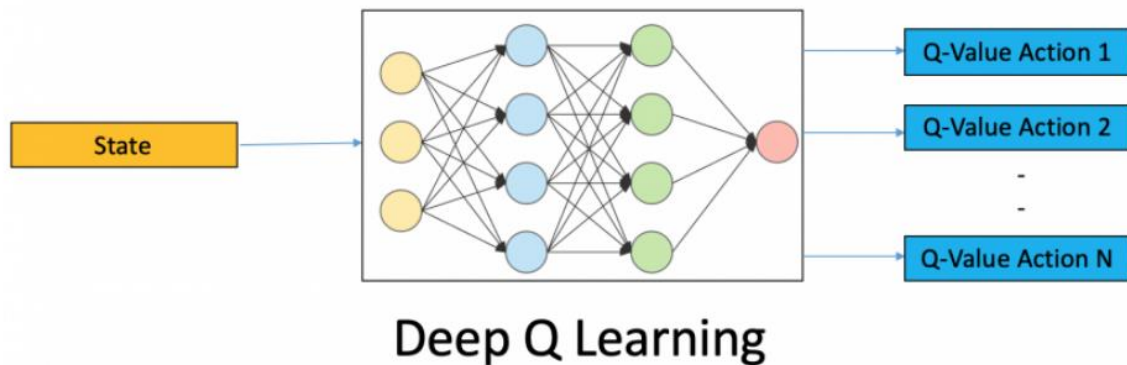


Figure 2-5. Q-Network [23]

There are two networks in Deep Q-learning, Q-Network and Target-Network. The training sets comprise numerous transitions, and they can be represented as $[s, a, r, s']$. Figure 2-6 illustrates the training procedure of Q-network. Q-Network and Target-Network take the current state s and the next state s' as inputs, respectively. Q-Network's parameters will be updated after calculating loss between outputs of the two networks. After several epochs, Target-Network takes Q-Network's parameters.

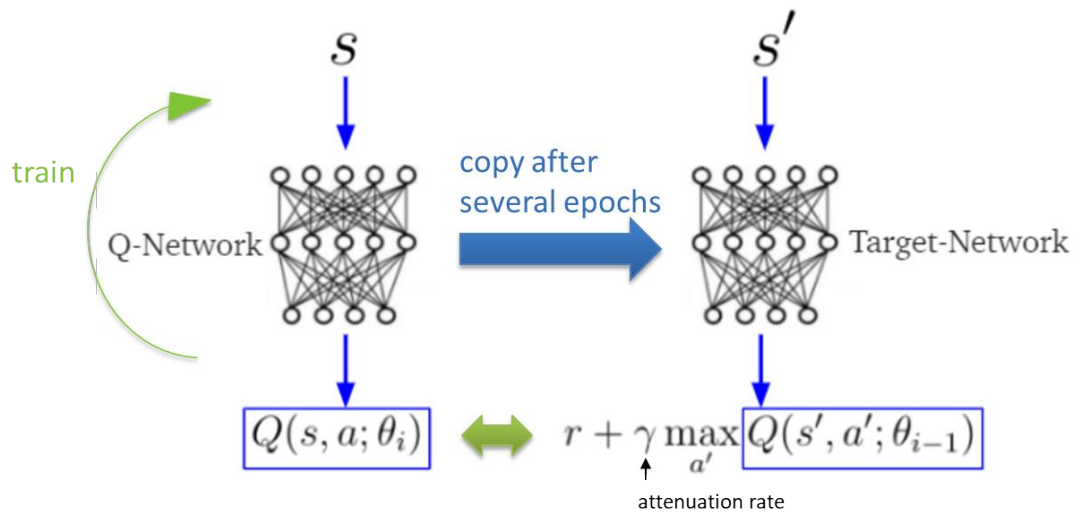


Figure 2-6. Training of Q-Network [24]

In DQN, Target-Network decides both action and reward which causes overestimation. Hence, in double deep Q-learning (DDQN), Q-Network takes the responsibility for action-decision, as depicted in Table 2-1.

Table 2-1. Action and reward decision

	action	reward
DQN	Target-Network	Target-Network
DDQN	Q-Network	Target-Network

Therefore, the formula for loss calculation $r + \gamma \max Q(s', a'; \theta_{i-1})$ is modified to $r + \gamma Q(s', \text{argmax} Q(s', a; \theta_i); \theta_{i-1})$ in DDQN.

Chapter 3 Related Works

DVFS stands out as one of the most promising approaches for GPU energy-saving as discussed in Chapter 1. In this chapter, we will discuss several studies related to our methodology.

3.1 Definition of compute-intensive, memory-intensive and hybrid [13]

Jiao et al. characterized kernel into compute-intensive, memory-intensive and hybrid by computing the rate of issuing instruction (R_{ins}) and global memory to computation cycle ratio (R_{mem}). The two values can represent ratio of compute-intensive and ratio of memory-intensive, respectively. They can be obtained by the following formulas:

$$R_{ins} = \frac{\text{number of computation instructions}}{\text{GPU execution time}} \quad (3-1)$$

$$R_{mem} = \frac{\text{number of global memory transactions}}{\text{number of computation instructions}} \quad (3-2)$$

The authors recorded both the number of global memory requests and transactions, because a single global memory request might result in multiple global memory transactions. They deducted the number of global memory requests from the total instruction count to get the number of computation instructions, while taking the number of global memory transactions to evaluate R_{mem} .

They executed three applications: matrix multiplication, matrix transpose, and fast Fourier transform, which were categorized into compute-intensive, memory-intensive and hybrid respectively on a NVIDIA Tesla GTX280 GPU. They presented that the relationship between performance and frequency varies on different applications. For example, the performance of compute-intensive application is linearly related to core

frequency but not varies when memory frequency is scaled. Furthermore, it gets the most power-efficiency when running at the highest core frequency and the lowest memory frequency. In contrast, the performance of the memory-intensive one is positively related to memory frequency but not affected by the core frequency, and it gets the most power-efficiency when running at the lowest core frequency and the highest memory frequency. The performance of hybrid application is linearly related to the core frequency, and the slope is determined by the memory frequency.

3.2 Shifting of compute-bound and memory-bound [21]

João et al. stated that the commonly used binary classification for kernels, compute-bound and memory-bound, may change under different frequency settings. Figure 3-1(a) presents the variation of relative execution time of memory and compute operations for three different core/memory frequency setting. At frequency state (F_{C1} , F_{M1}), the execution of Mem1 and Comp1 instructions occur at the same time and finish their execution simultaneously. Nonetheless, if core frequency increases or memory frequency decreases such that $\frac{F_{M2}}{F_{C2}} < \frac{F_{M1}}{F_{C1}}$, there will be an interval that only Mem instructions are executed on the GPU. Since the performance is mainly determined by the latency of the memory operations in this interval, the application is memory-bound. On the contrary, if core frequency decreases or memory frequency increases such that $\frac{F_{M3}}{F_{C3}} > \frac{F_{M1}}{F_{C1}}$, the application will shift to compute-bound. Figure 3-1(b) shows the performance of four different kernels changes with the memory frequency scaling. Kernel 1 is memory-bound at the highest memory frequency (F_{Mem5}), so it will remain memory-bound at any other available memory frequency setting. By contrast, Kernel 4

remains compute-bound even when memory frequency is lower than F_{Mem2} ; therefore, it will be compute-bound at most available memory frequency settings.

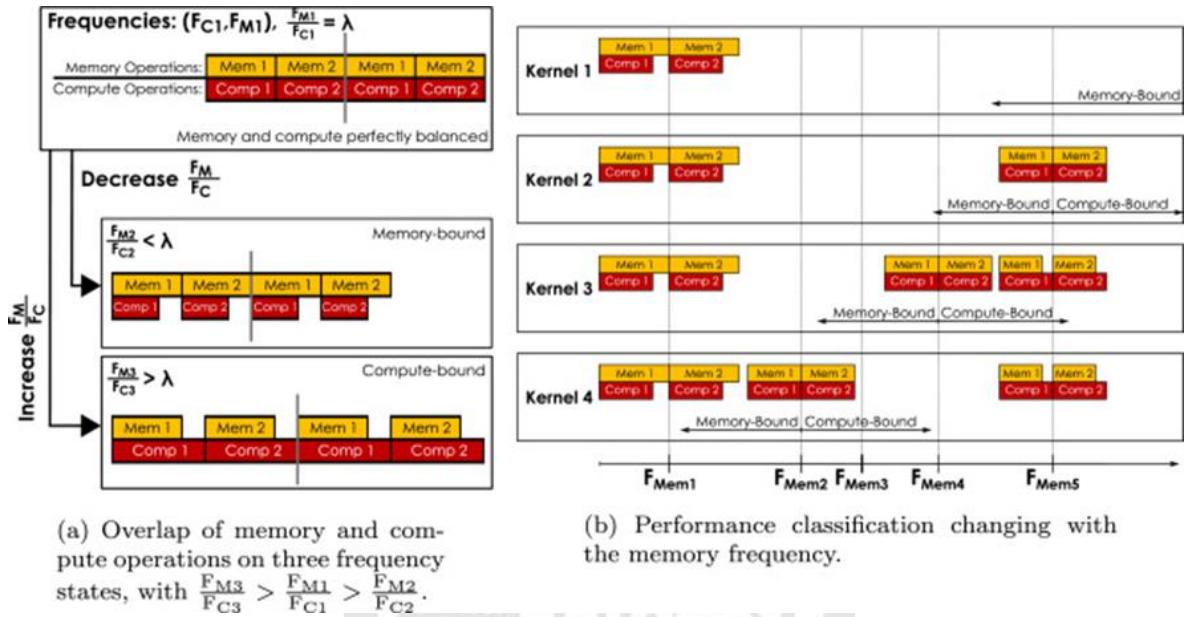


Figure 3-1. Examples of DVFS impact on performance [21]

The authors deemed that the impact of DVFS on performance is how the applications exploit graphic and memory resources. As presented in Figure 3-2, they grouped kernels into three classes based on their Peak Graphic Utilization and DRAM Utilization. Applications in Class TA are memory-bound at all memory frequency levels, so their performance is mainly determined by memory frequency. In contrast, the performance of applications in Class TC is dominated by core frequency. Most applications belong to Class TB, since their performance depends on both core and memory frequencies.

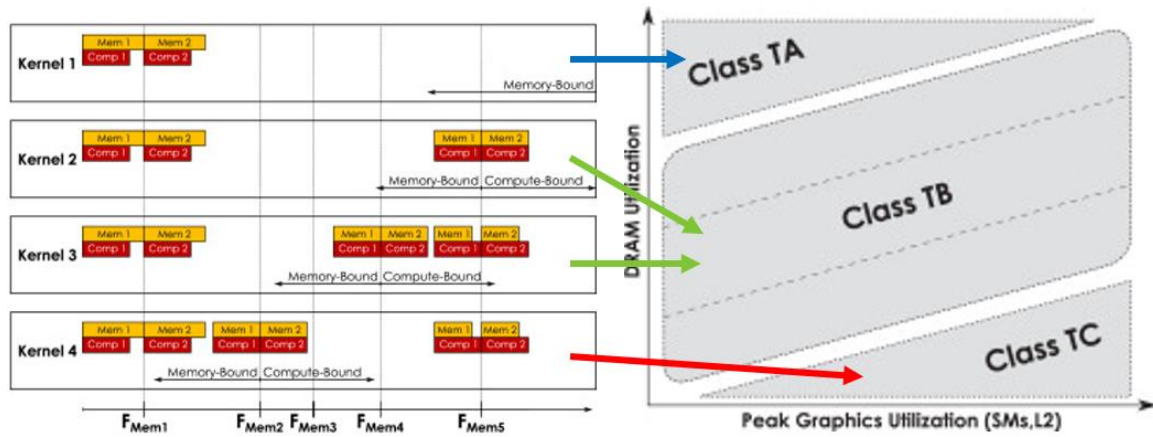


Figure 3-2. DVFS impact on performance classification [21]

GPU power is the summation of graphic power and memory power which can be expressed as Equation (3-3).

$$P_{GPU} = P_{Graphics}(F_{core}, V_{core}) + P_{Memory}(F_{mem}, V_{mem}) \quad (3-3)$$

Therefore, the authors also divided kernels into four classes based on the same parameters.

3.3 Contention reduction [16]

Sethia et al. observed that most of the GPU applications are bottlenecked by the number of compute resources, available memory bandwidth or limited data cache size. They proposed a runtime system which is called *Equalizer* that modulates the number of concurrent threads, SM and DRAM frequencies to balance kernels requirements and resource provided by hardware.

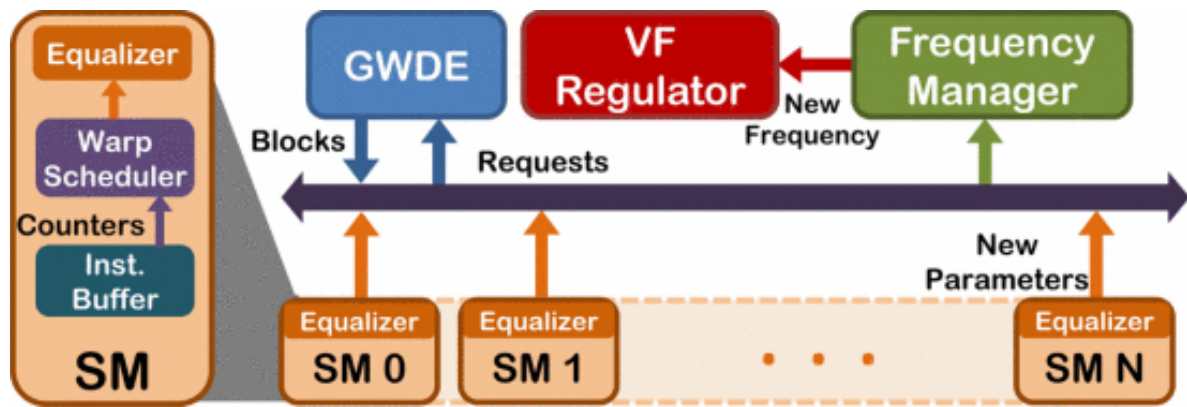
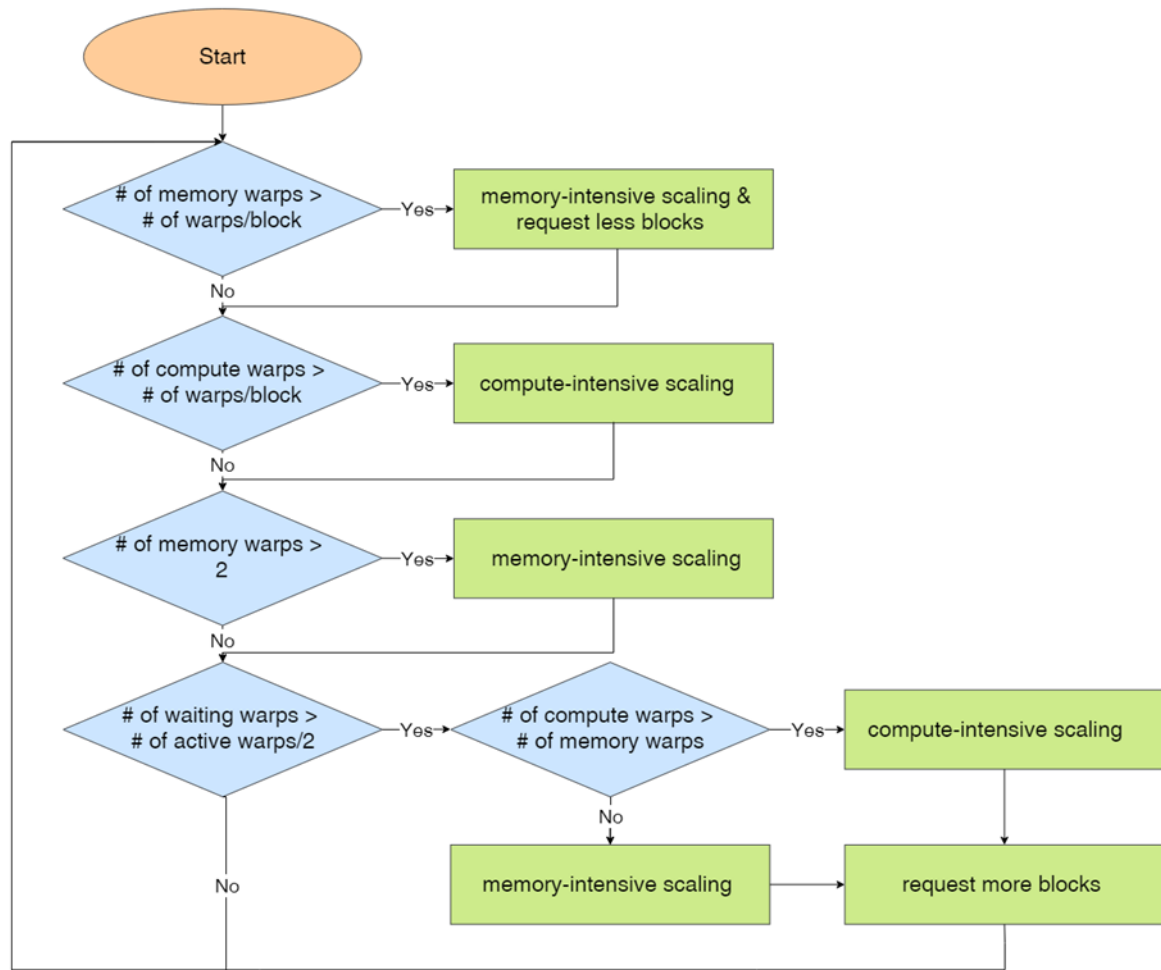


Figure 3-3. Equalizer overview [16]

Figure 3-3 shows the interaction among Equalizer and other GPU components. Every SM is embedded with an Equalizer, which continuously monitors four values: 1) the number of warps that are not paused or unaccounted (active warps), 2) the number of warps that cannot be executed because the scoreboard has not made the operands available (waiting warps), 3) the number of warps ready to be issued to the memory pipeline (memory warps), and 4) the number of warps ready to be issued to the arithmetic pipeline (compute warps). Based on the four values, Equalizer requests more/less blocks and submits local voltage/frequency (VF) preference to Frequency Manager. Frequency Manager then makes a global decision by the majority rule.



Algorithm 3-1. Equalizer algorithm [16]

Algorithm 3-1 illustrates the decision process of Equalizer. If the number of memory warps are greater than the number of warps per block, it indicates that the SM is under heavy cache contention. The SM will decrease the number of blocks by requesting less block from GWDE and submit MemAction to Frequency Manager. On the contrary, if the number of compute warps are larger than the number of warps per block, the SM is definitely compute-intensive. Hence, it will submit CompAciton to Frequency Manager. Additionally, the authors consider that memory bandwidth may be saturated if there are more than two warps in most SM. Therefore, the SM will send MemAction to Frequency Manager. If the number of unready warps is more than the number of ready warps, which means GPU resources are not fully utilized, the SM will request more block from GWDE

and submit Action after comparing the number of compute warps and the number of memory warps.

The system can be switched between energy mode for energy-saving and performance mode for performance-boosting. The frequency scaling mechanism is presented in Table 3-1. For example, if the majority of SMs submits MemAction, which means kernels are memory-intensive, Frequency Manager will decrease frequencies of all SMs and DRAM in energy mode but increase frequency of DRAM in performance mode.

Table 3-1. Frequency scaling based on different objectives

Kernel	Objective	SM frequency	DRAM frequency
Compute-intensive	Energy	Maintain	Decrease
	Performance	Increase	Maintain
Memory-intensive	Energy	Decrease	Decrease
	Performance	Maintain	Increase

3.4 DDQN on GPU power management [25]

Syu adopted double deep Q-learning(DDQN) to predict whether GPU is compute-intensive or memory-intensive in next interval for frequency/voltage scaling. As shown in Figure 3-4, power management unit forwards features, from SM to DDQN agent for scaling decision. After this interval, the agent will obtain reward if its prediction is right and penalty if it is wrong.

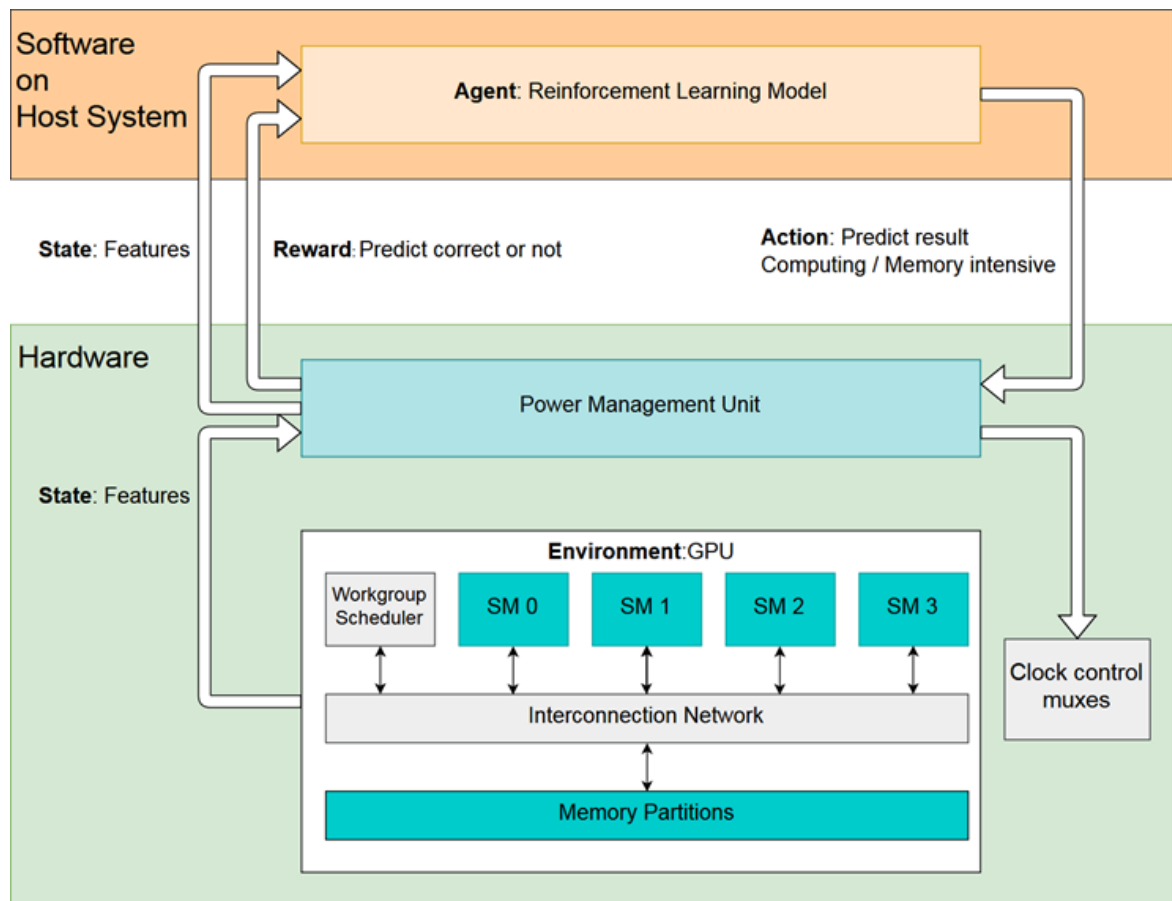


Figure 3-4. Interaction between DDQN agent and GPU [25]

Figure 3-5 shows the timeline of feature collections and tendency prediction. Power management unit samples GPU features in the first 5k cycles, and then transfers them to DDQN agent. It takes 7k cycles to predict the tendency for voltage and frequency scaling. The GPU continues running after voltage and frequency scaling until next sampling and learning after the second prediction.

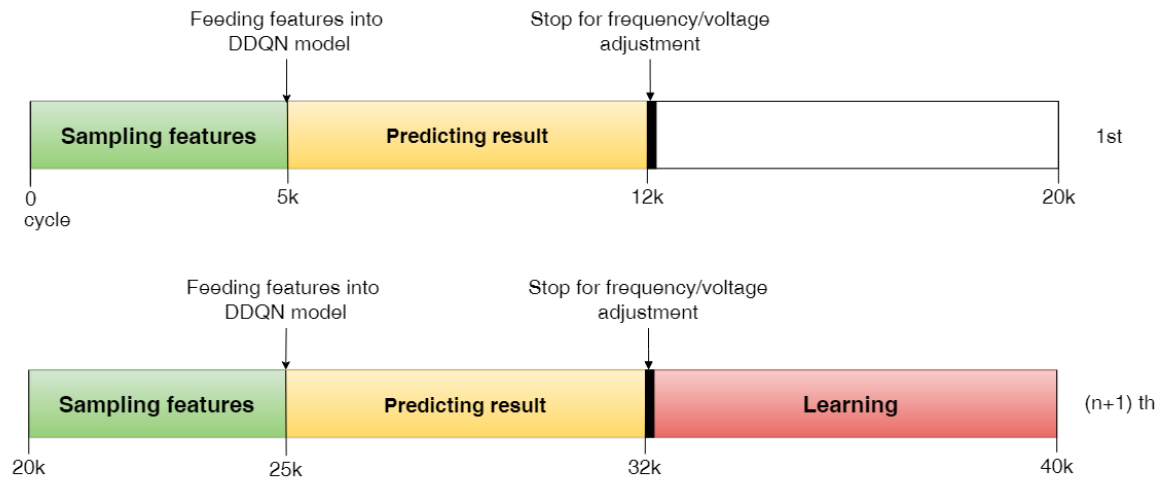


Figure 3-5. Timeline of feature collections and prediction [25]

The GPU scales up the SMs' frequency/voltage and scales down the memory frequency when the prediction result is compute-intensive. On the contrary, if the prediction result is memory-intensive, it scales down the SMs' frequency/voltage and scales up the memory frequency.

3.5 Summary of Related Works

Table 3-2 summarizes works described in Section 3.1-3.4.

- a. In related work I, Jiao et al. proposed formulas to calculate the ratio of compute-intensive and the ratio of memory-intensive which are widely adopted by succeeding works. They concluded that the core frequency at the highest level and the memory frequency at the lowest level is the most power-efficient setting for compute-intensive kernel. In contrast, it is much more power-efficient for memory-intensive kernel to operate at the lowest core frequency level and the highest memory frequency level.
- b. In related work II, João et al. indicated that the generally adopted binary classification of kernel, compute-bound and memory-bound, may vary with core or memory frequency scaling. They considered that characteristics of performance and

power shifting which caused by core/memory frequency, are related to graphic utilization and DRAM utilization.

- c. In related work III, Sethia et al. noticed most of the GPU applications are bottlenecked by the number of compute resources, memory bandwidth, and limited data cache size. They adjusted the number of concurrent threads to reduce cache contention in every SM. Furthermore, they balanced resources of SMs and DRAM by modulating their frequency.
- d. In related work IV, Syu employs DDQN to foretell GPU runtime tendency for frequency and voltage scaling. DDQN will scale up SMs frequency and scale down DRAM frequency when GPU is compute-intensive. In contrast, it scales down SMs frequency and scale up DRAM frequency when GPU is memory-intensive.

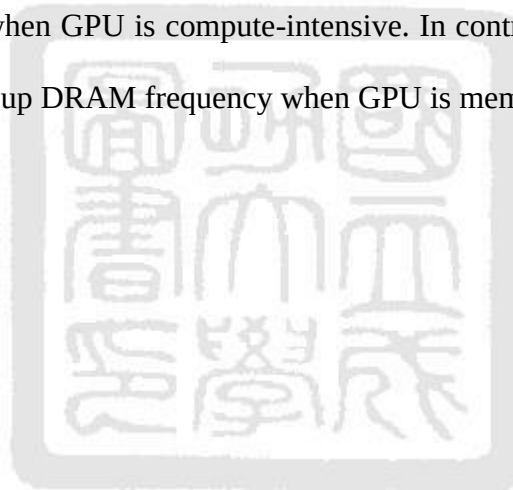


Table 3-2. Related works summary

	Related Work I	Related Work II	Related Work III	Related Work IV
Publication	CPSCoM'10	Parallel Computing'18	MICRO'14	LPHP lab
Platform	NVIDIA GeForce GTX 280 (hardware)	NVIDIA GTX Titan X and Titan Xp (hardware)	GPGPU-Sim (simulation)	CASLAB-GPUSim (simulation)
Benchmark suites	Self-designed	Parboil, Rodinia, SHOC, PolyBench and CUDA SDK	Rodinia and Parboil	PolyBench
Main idea	Select the most power-efficient frequency setting for compute-intensive and memory-intensive kernels, respectively	Kernels' characteristics (compute-bound and memory-bound) may shift along with frequency	Beside DVFS on core and memory, control block dispatching to lessen cache contention	Adopt DDQN to level up/down frequency/voltage of core and memory

Chapter 4 The Proposed GPU Power Management

Energy-efficiency is defined as performance/energy. Therefore, to improve energy-efficiency, we need to either increase performance or decrease energy consumption of GPU. We propose a Cache-Contention-Aware Algorithm described in Section 4.1 to boost GPU performance and DVFS based on DDQN, which will be depicted in Section 4.2 for energy saving.

4.1 Cache-Contention-Aware Task Dispatch Algorithm

We sample the number of memory tasks every 100k cycles and notice that memory-related tasks sometimes concentrate on several SMs which cause contention problem as shown in Figure 4-1.

In order to measure the severity of cache contention, we count the ratio of pending entries and the ratio of occupied entries every 20k cycle. We observe that up to 25% cache entries are pending for data from memory at peak when executing some benchmarks and all the cache entries are occupied during most of the execution as presented in Figure 4-2.

	without			
Cycle(k)	SM1	SM2	SM3	SM4
100	5	5	5	5
200	5	5	5	5
300	5	0	5	0
400	4	5	5	5
500	5	5	5	5
600	5	5	5	5
700	0	0	0	4
800	5	5	5	5
900	4	6	5	5
1000	5	2	5	0

Figure 4-1. Number of memory-related task in every SM

		2MM	3MM	GEM M	SYRK	2DCO NV	MVT	ATAX	BICG
<u>pending entries</u> total entries	Peak	20%	25%	25%	25%	14%	9%	6%	6%
	Ave.	3%	7%	7%	20%	5%	3%	6%	5%
<u>occupied entries</u> total entries	Peak	100%	100%	100%	100%	100%	100%	100%	100%
	Ave.	99%	87%	89%	90%	99%	56%	55%	55%

Figure 4-2. The severity of cache contention when executing different benchmarks

As Figure 4-3 illustrates, memory-related tasks concentrate on SM1 and SM2, which causes highly cache contention depicted in Related work III. If more tasks are dispatched to these SMs, the contention problem will be more severe as shown in Figure 4-4. This bottlenecked condition not only reduces GPU performance but also induces unnecessary power consumption of idle components.

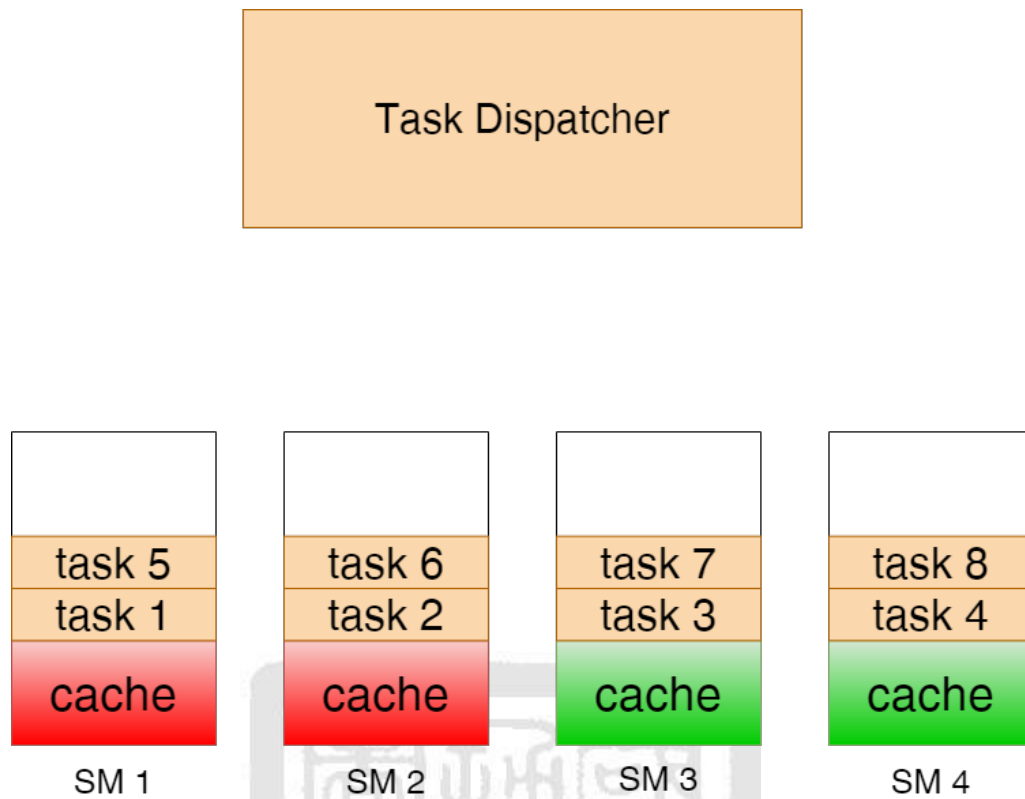


Figure 4-3. Several SMs with cache contention problem

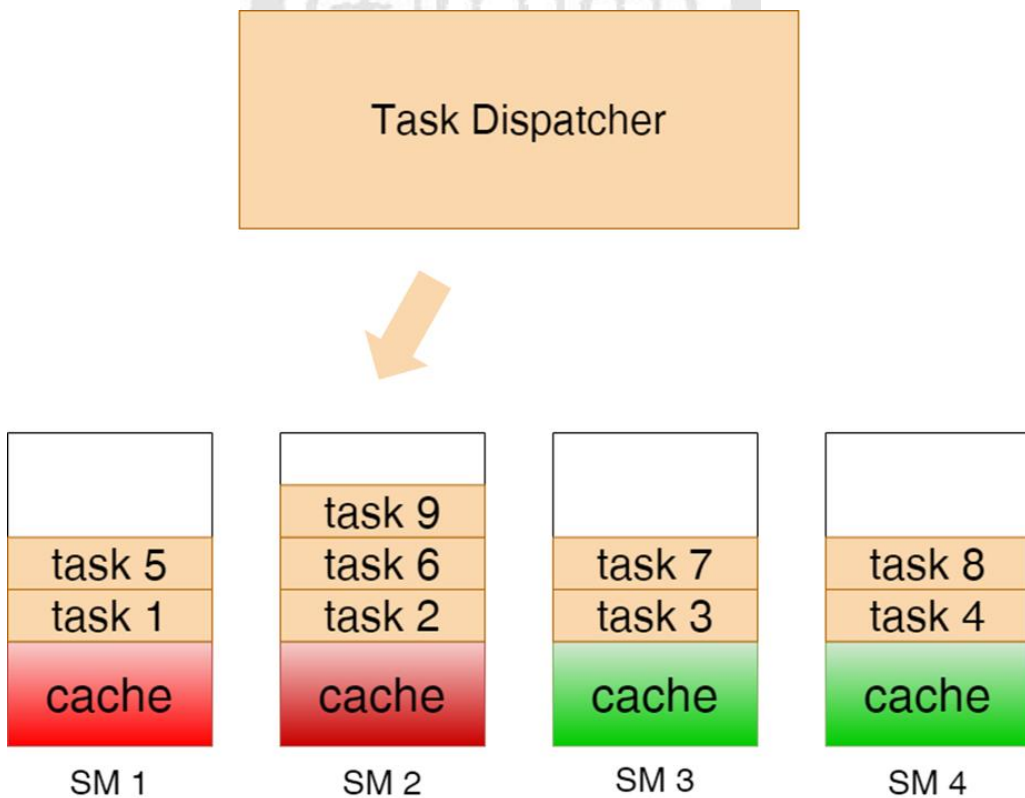


Figure 4-4. More severe contention in SM 2

Hence, we are motivated by the related work III to develop a dynamic Cache-Contention-Aware Task Dispatch Algorithm (CCATD).

Figure 4-5 shows the decision flow of SM. Every SM continuously collects three parameters in Instruction Buffer:

- 1) The number of memory warps: the number of warps which are ready to be sent to Load/Store Unit.
- 2) The number of unready warps: the number of warps which are not ready to be issued because of data hazard.
- 3) The number of ready warps: the number of warps which are ready to be issued.

They evaluate severity of cache contention and resource utilization with Equation (4-1 and Equation (4-2

$$\text{cache contention index} = \frac{\text{number of memory warps}}{\text{number of warps/task}} \quad (4-1)$$

$$\text{unsaturation index} = \frac{\text{number of unready warps}}{\text{number of ready warps}} \quad (4-2)$$

If the cache contention index is greater than 1, it means the SM has severe cache contention; therefore, the SM will request to stop adding tasks, i.e., equivalently to lessen the number of tasks from Task Dispatcher to reduce the contention. When the cache contention index ≤ 1 and the unsaturation index is greater than a threshold β , resources in the SM are not fully utilized. Therefore, the SM will request more tasks from Task Dispatcher to increase resources utilization. β is an empirical value and will be determined in Experiment 1.

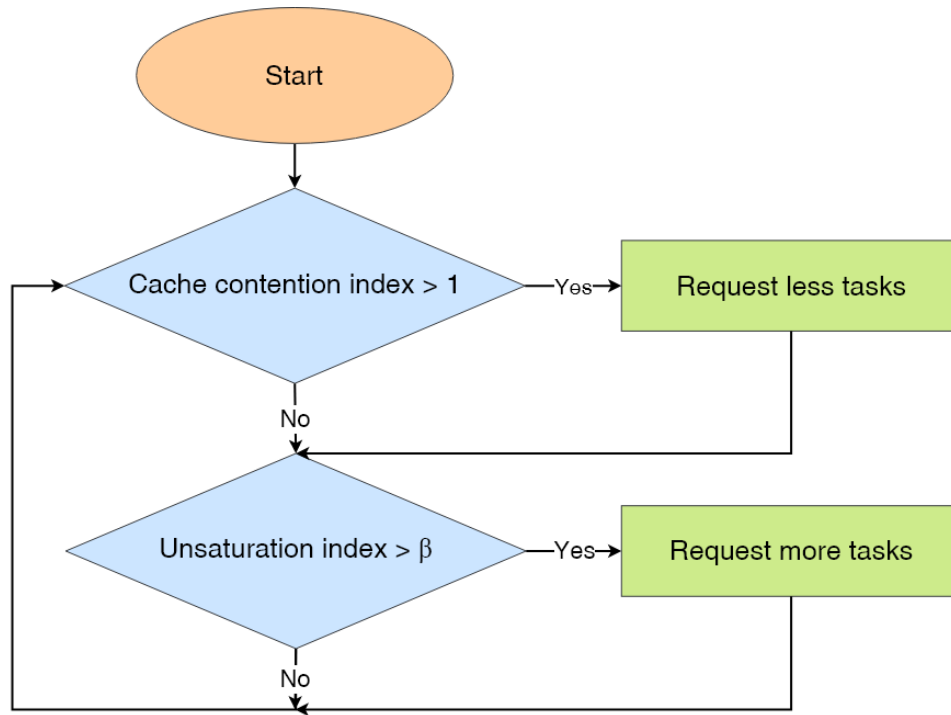


Figure 4-5. CCATD (SM side)

The Dispatcher in the CASLAB-GPUSim does not consider the contention problem in every SM before dispatch. We modify its Dispatcher algorithm as shown in Figure 4-6. In the original design, Task Dispatcher assigns a new task to a SM if it has empty entry. In our proposed design, the Dispatcher takes SM's request into account. We present the request from a SM to the Dispatcher with a green link in Figure 4-7.

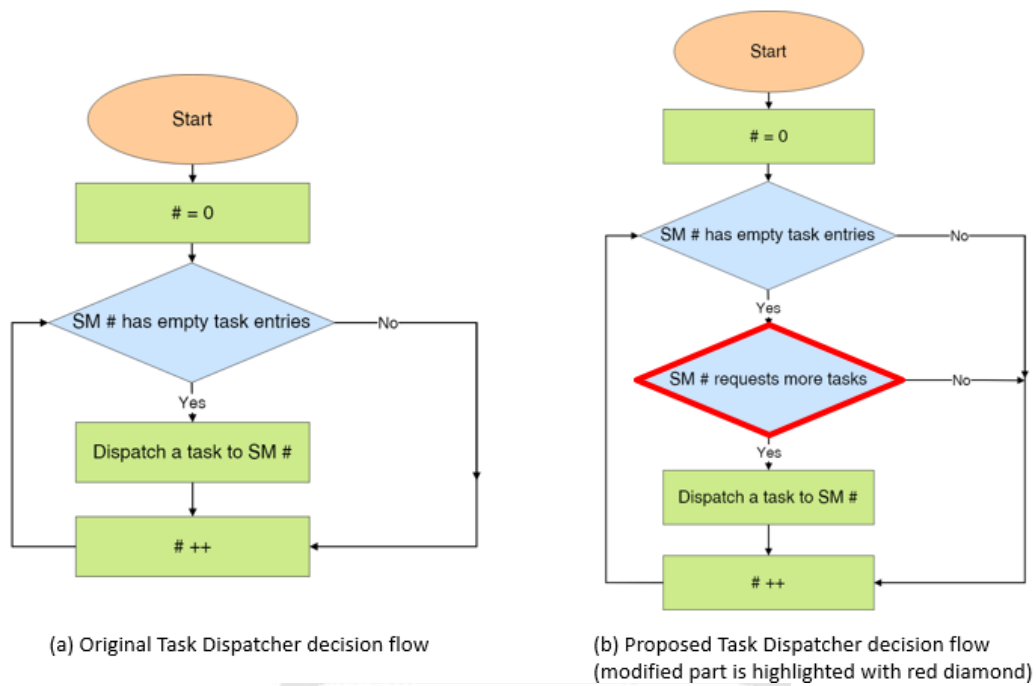


Figure 4-6. CCATD (Task Dispatcher side is modified from (a) to (b))

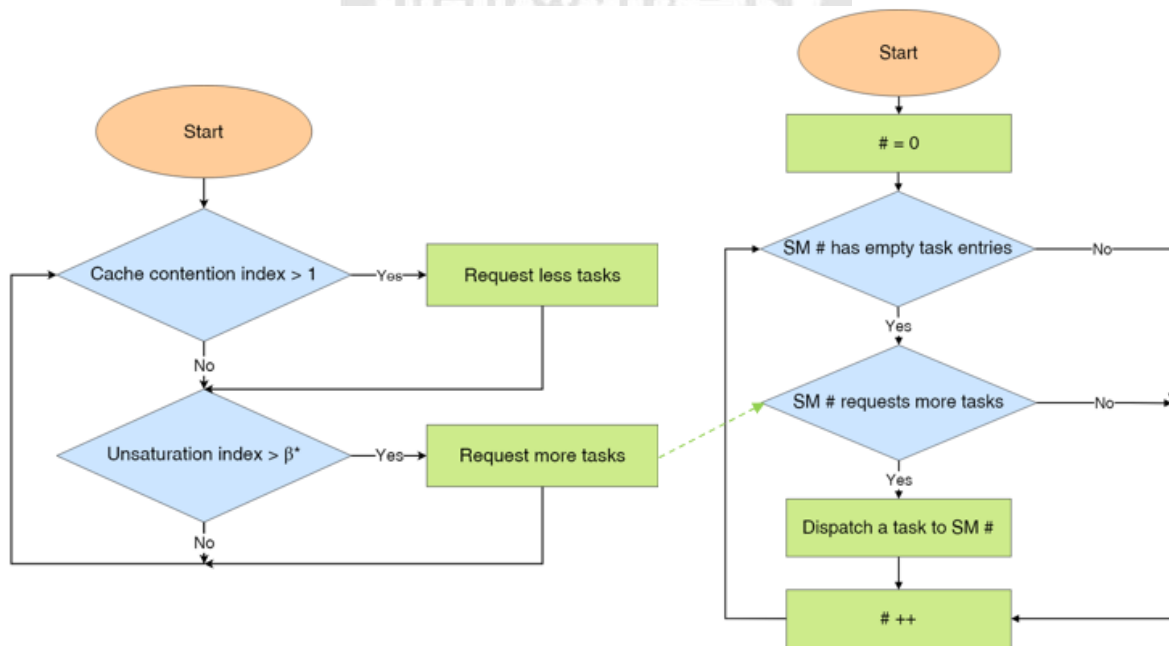


Figure 4-7. Interaction between SM and Task Dispatcher

Figure 4-8 shows one possible interaction of SMs and Task Dispatcher. SM1 and SM2 request less tasks from task dispatcher, the memory-related task will be dispatched to SM3. Because SM3 is embedded with more space in its cache, this decision will improve GPU throughput.

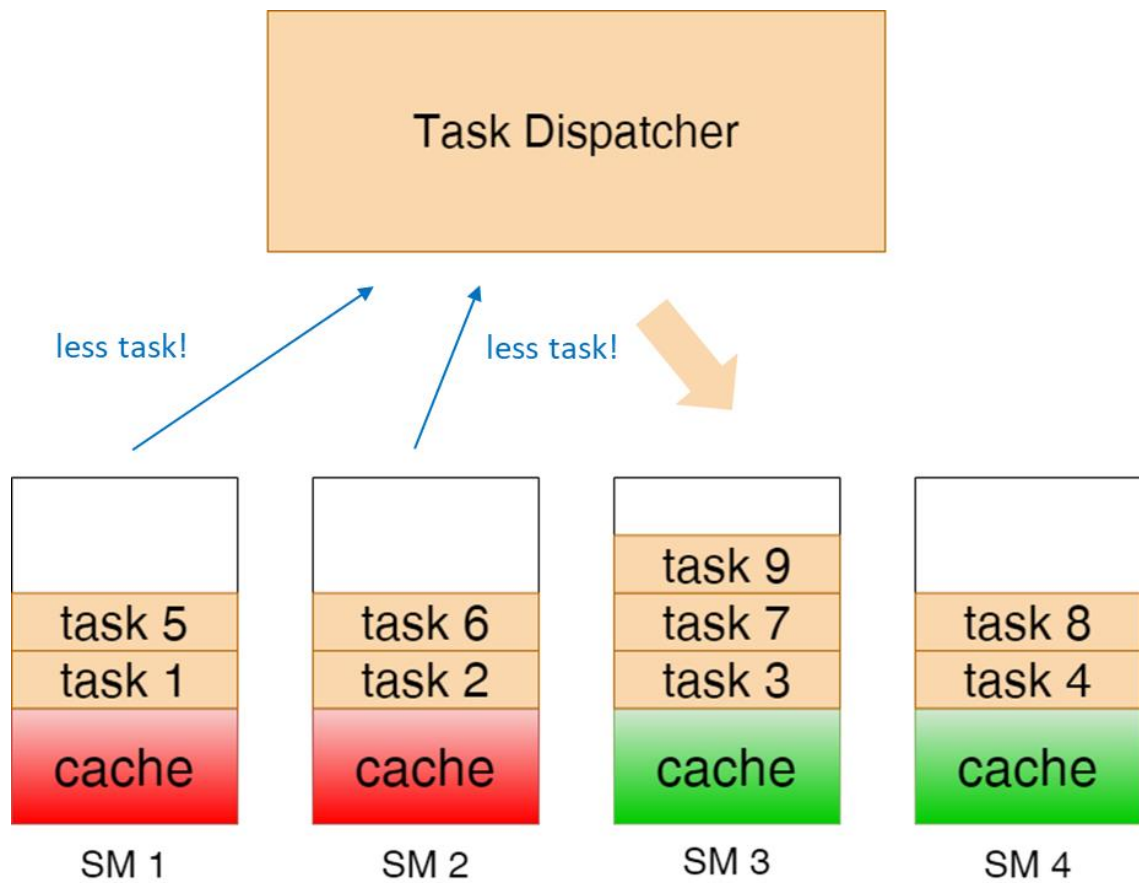


Figure 4-8. SMs with cache contention request less tasks

After SM1 and SM2 relieve from cache contention, and not fully utilize their hardware resource, they request more tasks as presented in Figure 4-9.

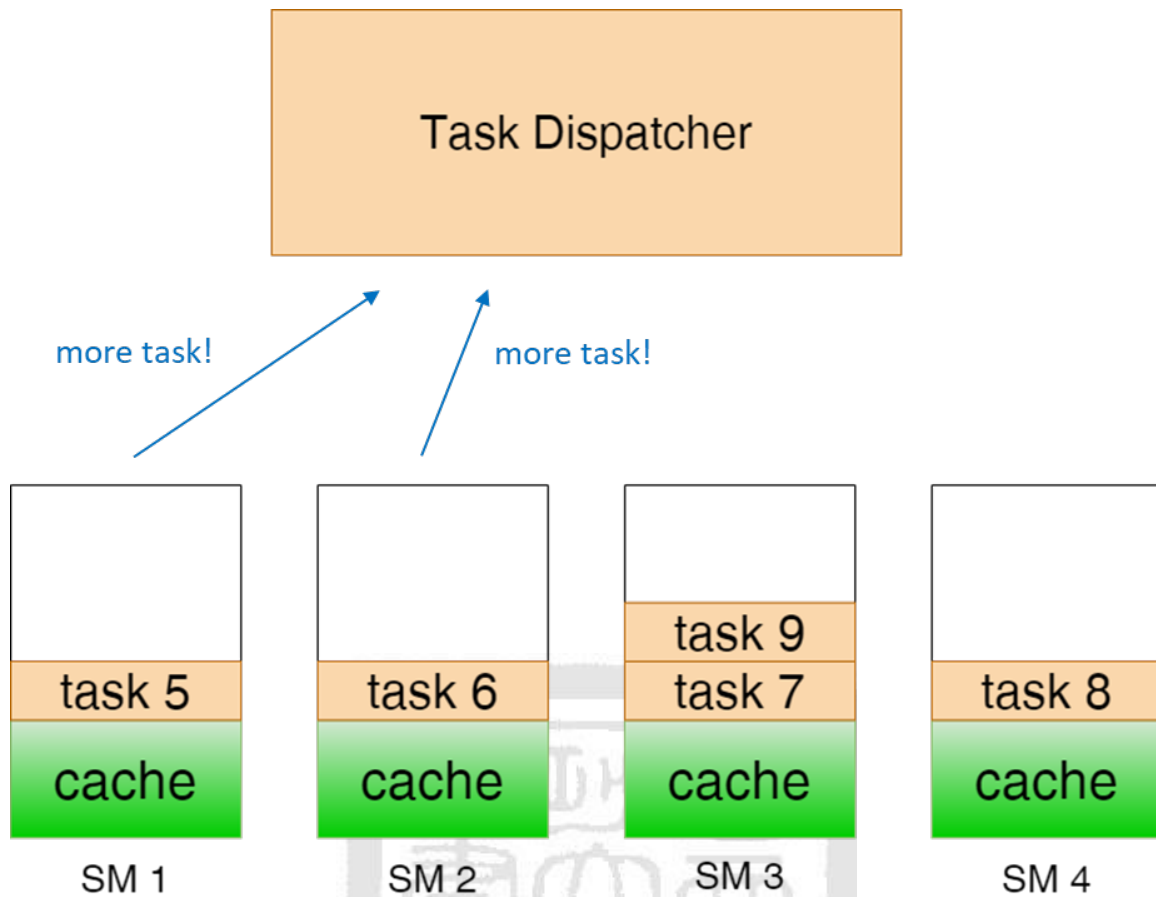


Figure 4-9. Under-utilized SMs request more tasks

4.2 DVFS based on DDQN

As illustrated in related work III, if every SM sends request to DRAM, the bandwidth will be saturated. However, a single memory warp in every SM does not indicate pressure on bandwidth, because this warp may be hit in L1 cache. Therefore, they assume that if there are two memory warps in each SM, the memory bandwidth will be saturated. It is a common phenomenon by observing several benchmarks in PolyBench which means decreasing SMs frequency may reduce bandwidth saturation and boost GPU performance.

Nevertheless, it is difficult to decide the suitable frequency for all runtime situations in all type of kernels. As DDQN has been proved as an effective way for power

management in related work IV, we adopt it to decide whether to level up or level down SM/DRAM frequency. Decreasing SM frequency not only lessens bandwidth saturation, but also reduces SM power consumption. In order to maintain GPU performance with frequency lower than default setting, we take instruction per seconds (IPS) as DDQN reward. Figure 4-10 shows the interaction between DDQN and GPU. Power management unit collects features from GPU, and then transmit them to DDQN agent. Then, GPU adjusts frequency/voltage to comply with the decision from DDQN agent. After one interval, the power management unit take IPS as a reward for DDQN agent.

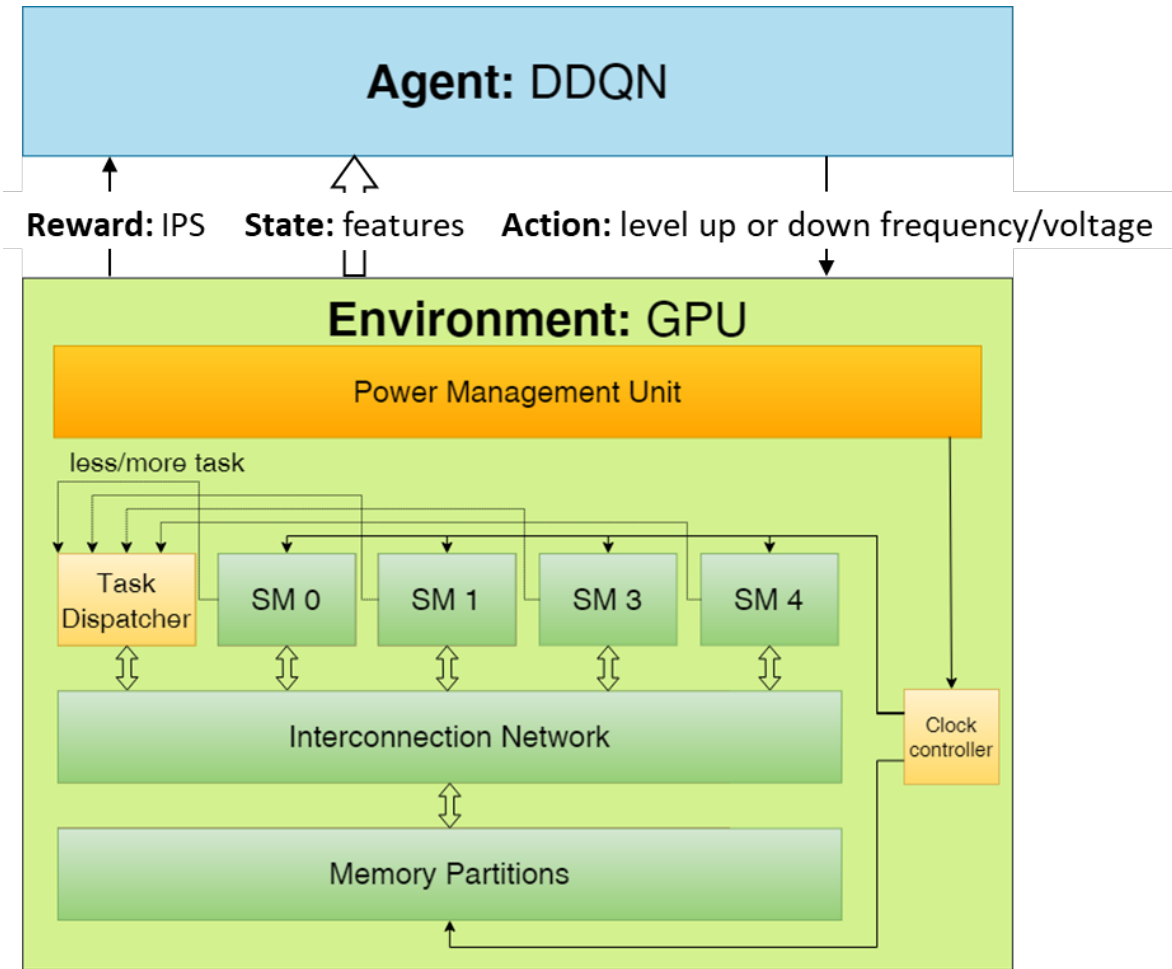


Figure 4-10. System architecture

4.2.1 GPU Features

Block filled with green color in Figure 4-11 are embedded with feature monitors.

There are three types of features:

- 1) Decreasing rate: defined by the following equation

$$\begin{aligned} \text{Decreasing rate} & \quad (4-3) \\ &= \frac{\text{occurrences of decreasing during this interval}}{\text{interval length}} \end{aligned}$$

- 2) Instruction per cycle (IPC): obtained by the following equation

$$IPC = \frac{\text{number of issued instructions during this interval}}{\text{number of cycles during this interval}} \quad (4-4)$$

- 3) Transient ratio for compute-intensive and memory-intensive ($\hat{R}_{ins}$ and $\hat{R}_{mem}$): estimated using the following equation. We adopt the absolute value as interval length because the cycle period changes with frequency scaling

$$\hat{R}_{ins} = \frac{\text{number of compute instructions during this interval}}{\text{interval length(ns)}} \quad (4-5)$$

$$\hat{R}_{mem} = \frac{\text{DRAM read and write count during this interval}}{\text{number of compute instructions during this interval}} \quad (4-6)$$

We collect decreasing rate of the following parameters:

- 1) The number of compute/memory/branch/unready warps in Instruction Buffer
- 2) The length of a queue in
 - a、Load/Store Unit
 - b、MSHR
 - c、compute warps/memory warps in Operand Collect Unit

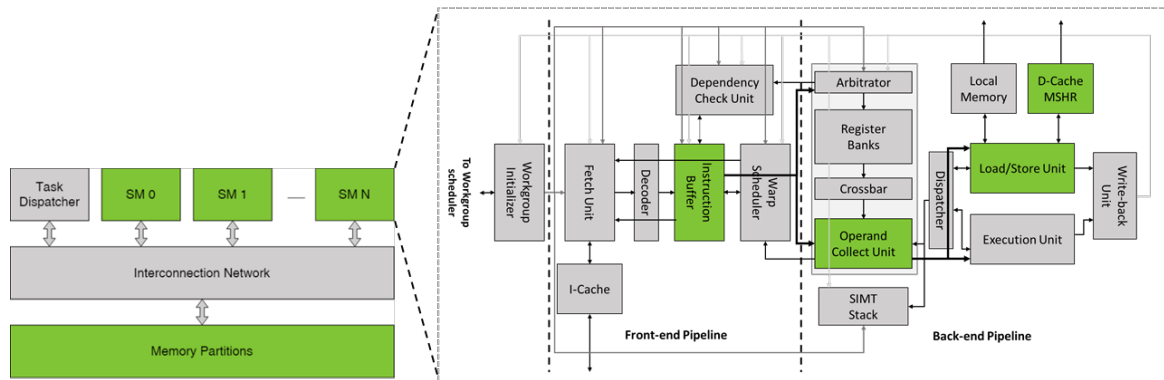


Figure 4-11. Blocks with feature monitors monitor (marked by green color)



Chapter 5 Experimental Results and Analysis

In this chapter, we will detail the experiment methodology and setup in Section 5.1. In Section 5.2, we present our experiment results

5.1 Environment Setup

We detail the platform setting in Table 5-1 and DDQN hyperparameter setting in Table 5-2.

Table 5-1. Platform setting

Number of SM	4
Platform	CASLAB-GPUSim ESL platform (SystemC) [26, 27]
Benchmark suite	PolyBench
SM frequency/voltage level	66.6MHz - 200MHz / 0.72V - 0.9V
DRAM frequency level	66.6MHz - 400MHz
Warp scheduler policy	Loose round robin

Table 5-2. DDQN hyperparameter

Network architecture	Fully-connected ● # of input neurons: 11 ● # of hidden neurons: 90 ● # of output neurons: 2
Learning rate	0.001
attenuation rate	0.95
Minimum exploration rate	0.01

5.1.1 GPU Virtual Platform Power Model

We build a register file and a SRAM based on the size of the register file and the L1 Cache respectively by 40nm ARM memory compiler for read/write power, and refer to Micron DRAM spec. for power information of DRAM [28]. As for other modules, the procedure is depicted in Figure 5-1. We build RTL module based on ESL module behavior, verified by assuring outputs are identical with the same inputs. We synthesized these RTL modules for power measurement on Prime-time PX of 6 benchmarks. According to Figure 5-2, the average error rate is 2.38%.

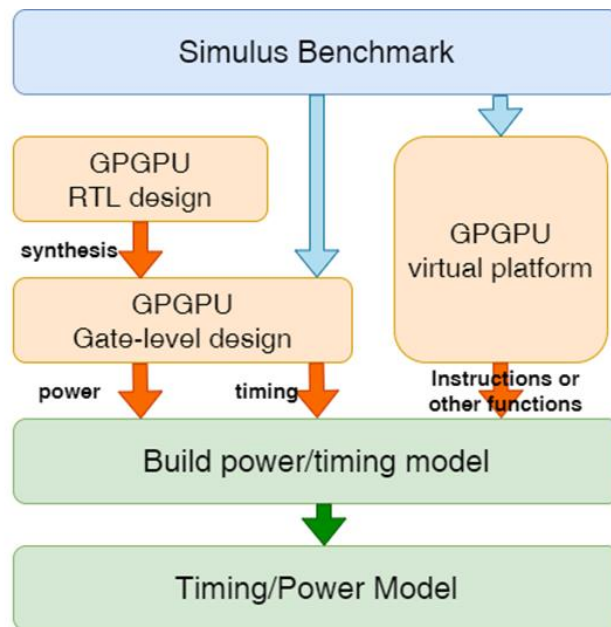


Figure 5-1. Flow for power model [29]

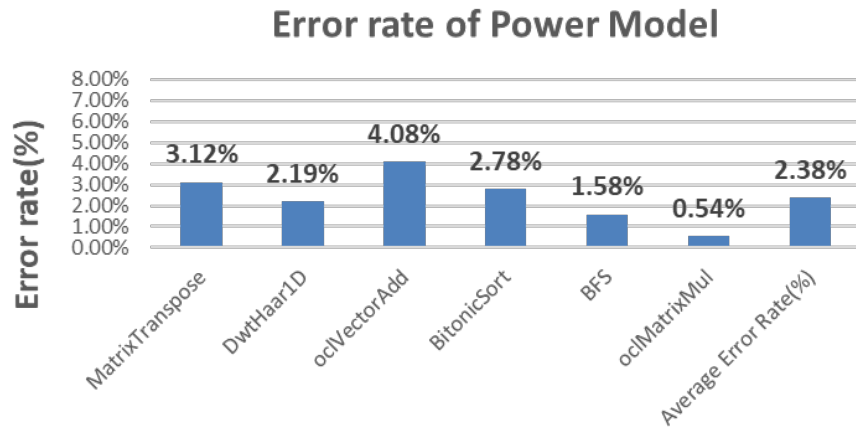


Figure 5-2. Error rate of power model [29]

5.2 Experimental results

5.2.1 Experiment 1

To determine β in CCATD, we select four benchmarks with cache contention issues, 2MM, 3MM, GEMM, and SYRM. We execute them in different settings to calculate average energy and performance. The result is presented in Figure 5-3 and Figure 5-4.

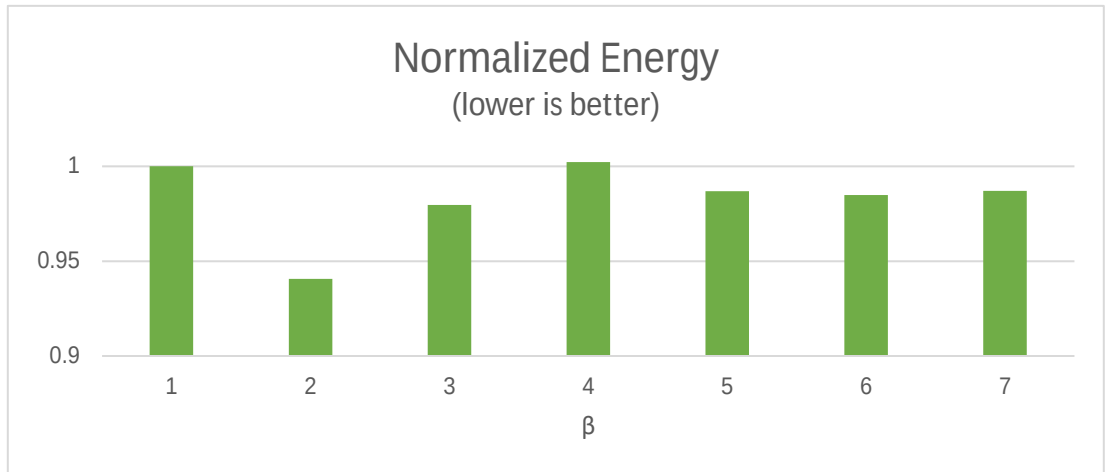


Figure 5-3. Average energy in different β settings

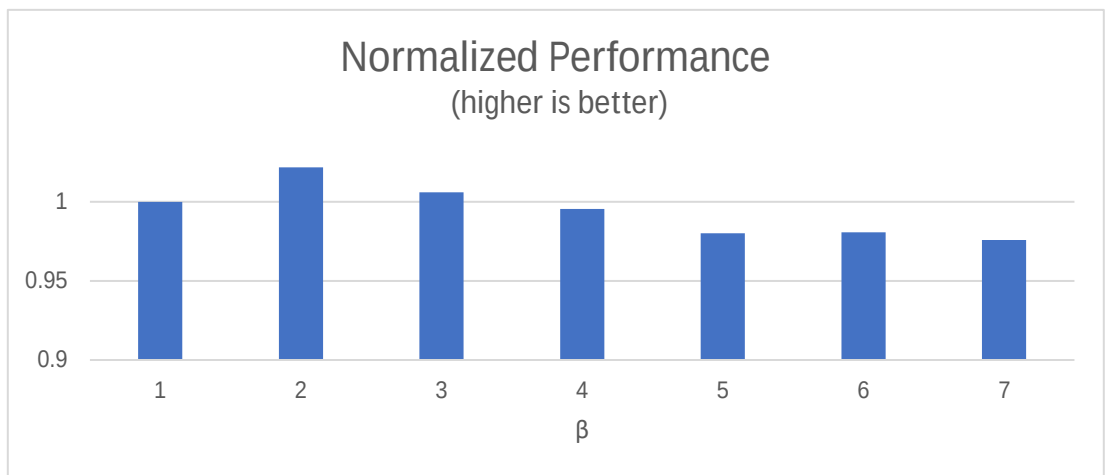


Figure 5-4. Average performance in different β settings

Since the GPU consumes the least energy and achieve the best performance when β is 2, we set β as 2 in the following experiments.

5.2.2 Experiment 2

In this experiment, we compare energy and performance of proposed design with those of other methods. We adopt timeline in related work IV, which is shown in Figure 5-5.

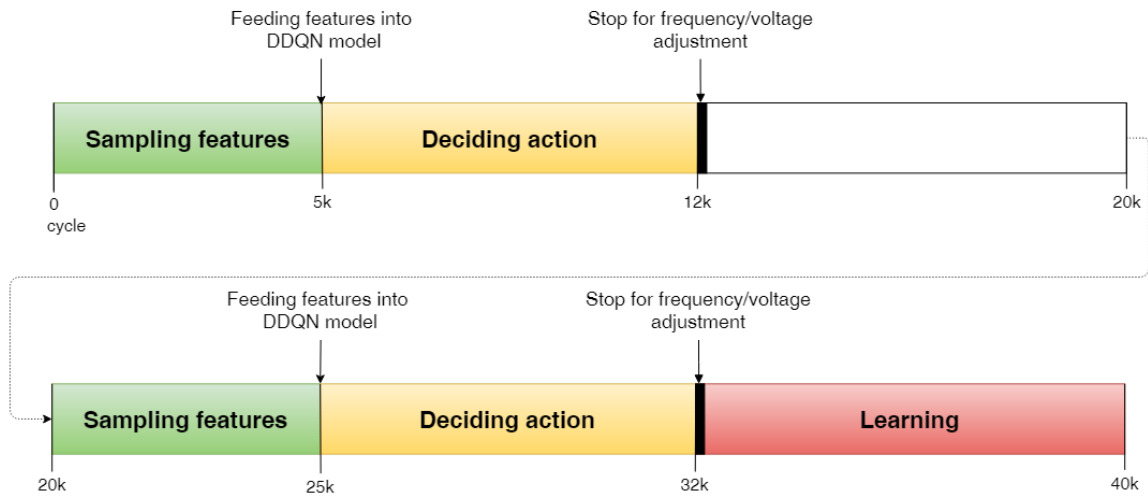
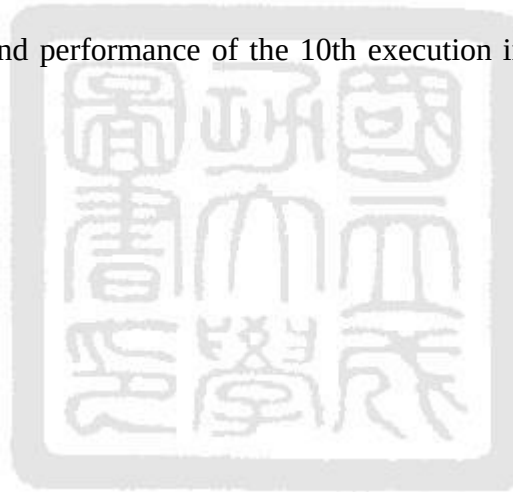


Figure 5-5. Timeline in experiment 2 and experiment 3

We train model which fits each benchmark by running each of them 10 times after reset. We show energy and performance of the 10th execution in Figure 5-6 and Figure 5-7, respectively.



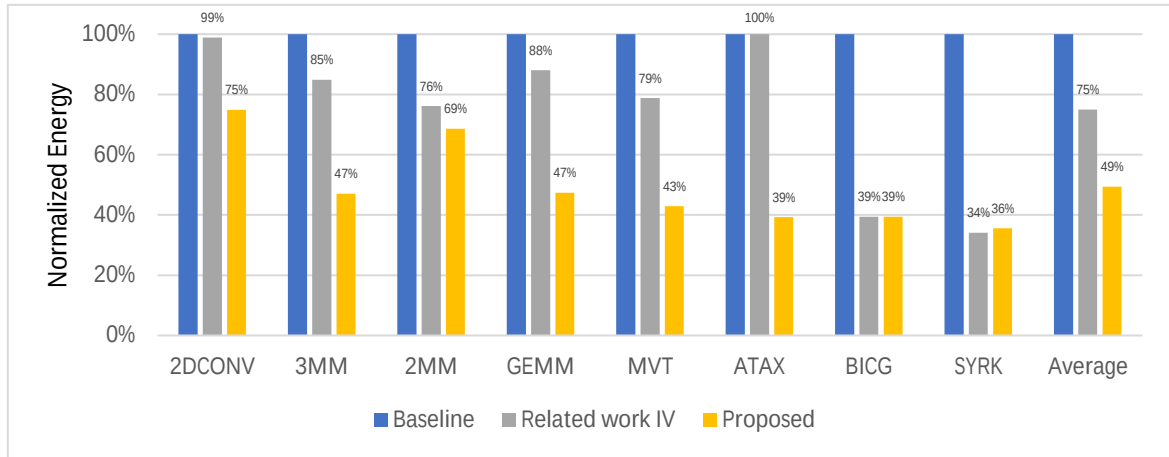


Figure 5-6. Energy comparison in different benchmarks

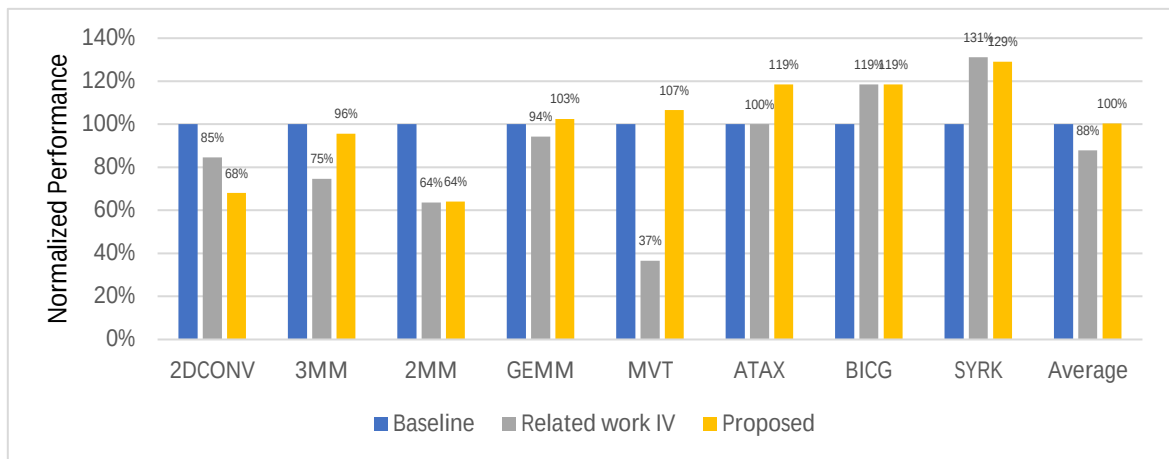


Figure 5-7. Performance comparison in different benchmarks

Our proposed design achieves more energy-saving than related work IV in most applications (2D CONV, 3MM, 2MM, GEMM, MVT, ATAX) with less performance degradation (except for 2D CONV). Comparing to 25% energy-saving and 12% performance loss in related work IV, our proposed design achieves 51% energy-saving with no performance loss on average.

5.2.3 Experiment 3

We adopt the metric used in related work I to evaluate ratio of compute-intensive (R_{ins}) and ratio of memory-intensive (R_{mem}) of all benchmarks. We reset register A and register B to 0 in the beginning. We add the number of compute instructions in

Instruction Buffer of all SMs to register A every cycle and add 1 to register B when DRAM is accessed as shown in Equation (5-1 and Equation (5-2. We divide the value of register A by total execution time (ns) for R_{ins} and divide register B by register A for R_{mem} as presented in Equation (5-3 and Equation (5-4.

$$A(t) = A(t - 1) + \# \text{ of compute instructions of all SMs} \quad (5-1)$$

$$B(t) = \begin{cases} B(t - 1), & \text{DRAM is not accessed} \\ B(t - 1) + 1, & \text{DRAM is accessed} \end{cases} \quad (5-2)$$

$$R_{ins} = \frac{A(t)}{\text{total execution time(ns)}} \quad (5-3)$$

$$R_{mem} = \frac{B(t)}{A(t)} \quad (5-4)$$

Depending on Figure 5-8, we can categorize 2MM, 3MM, GEMM, 2DCONV into compute-intensive, and ATAX, BICG, MVT, SYRK into memory-intensive.

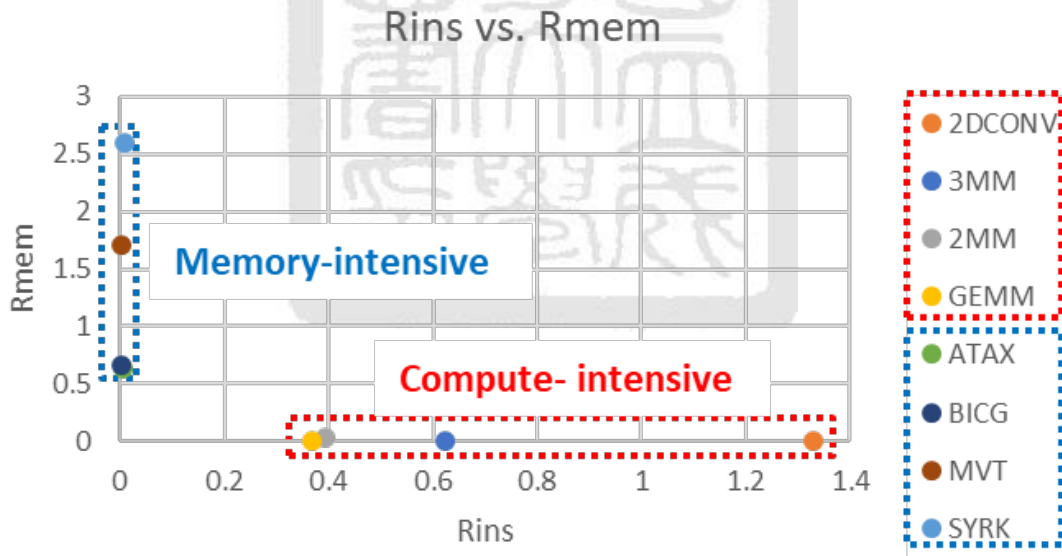


Figure 5-8. R_{ins} and R_{mem} of all benchmarks

In order to test the model adaptability, we design four series:

1. Series1: 2MM → ATAX → 3MM → BICG → GEMM → MVT → 2DCONV → SYRK
2. Series2: 2MM → 3MM → ATAX → BICG → GEMM → 2DCONV → MVT → SYRK
3. Series3: 2MM → 3MM → GEMM → ATAX → BICG → MVT → SYRK → 2DCONV

4. Series4: 2MM→3MM→GEMM→2DCONV→ATAX→BICG→MVT→SYRK

We fill the training batch with different number of characteristics from compute-intensive kernels to test whether the following memory-intensive kernels gain the same energy-saving. We train model which fits each series by running the same series 10 times after reset. Figure 5-9 and Figure 5-10 present energy in every benchmark of the 1st and 2nd execution. We observe that the model of series2 and series4 cannot achieve energy-saving in several benchmarks in the beginning. Nonetheless, after it collect features of all benchmarks, it reduces the energy in these benchmarks and acts nearly the same in the following execution. We show energy in every benchmark and their contribution to total energy of the 10th execution in Figure 5-11 and Figure 5-12, respectively. Furthermore, The performance in every benchmark and their contribution to total execution time is presented in Figure 5-13 and Figure 5-14, respectively.

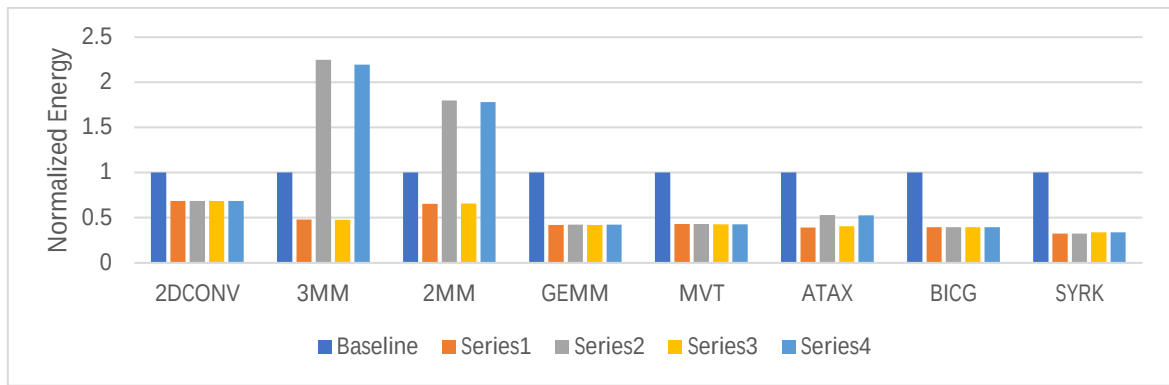


Figure 5-9. Normalized energy in every benchmark (1st execution)

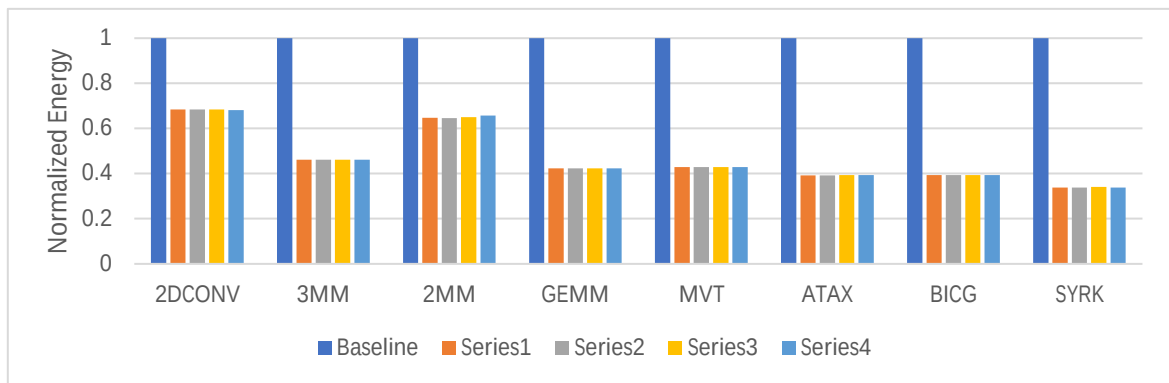


Figure 5-10. Normalized energy in every benchmark (2nd execution)

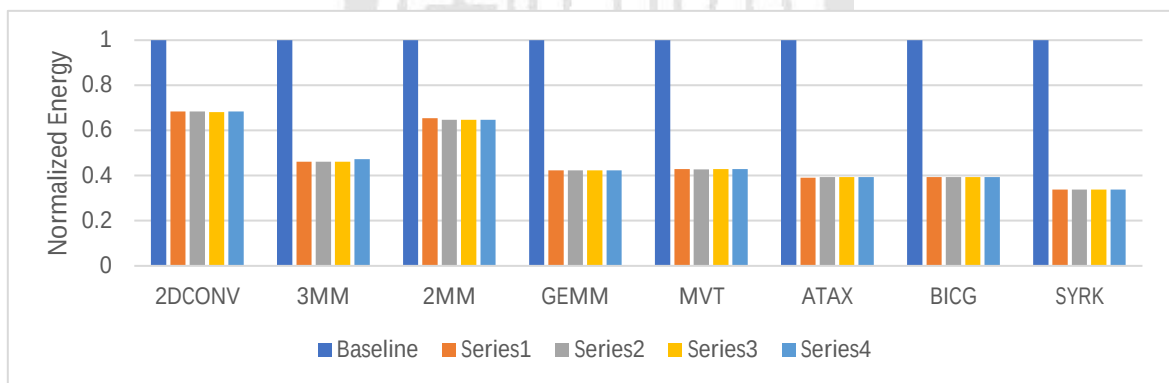


Figure 5-11. Normalized energy in every benchmark (10th execution)

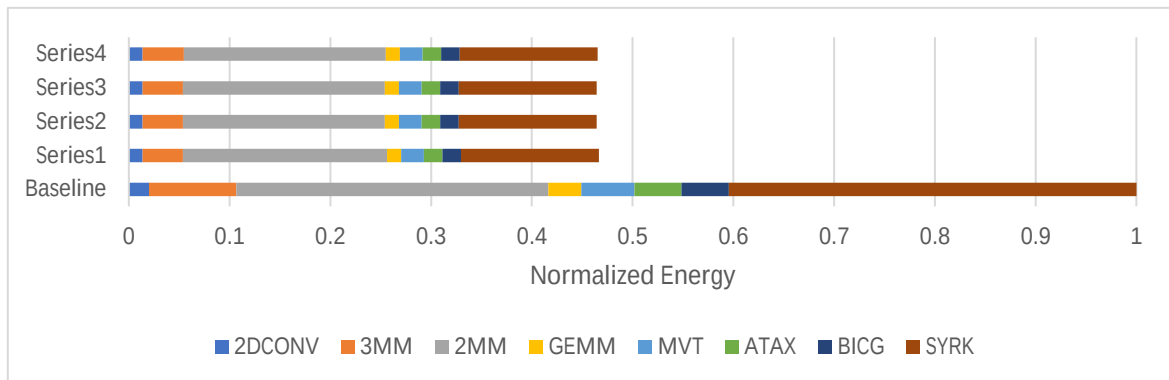


Figure 5-12. Normalized energy in every series (10th execution)

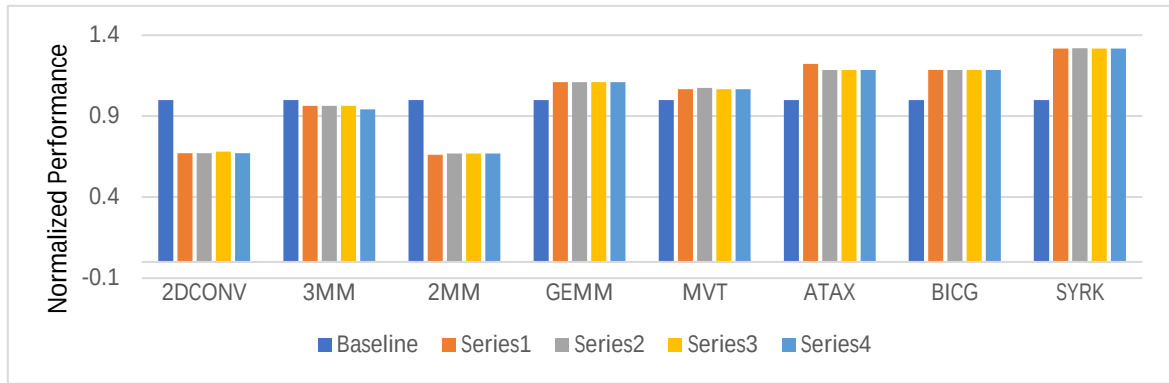


Figure 5-13. Normalized performance in every benchmark

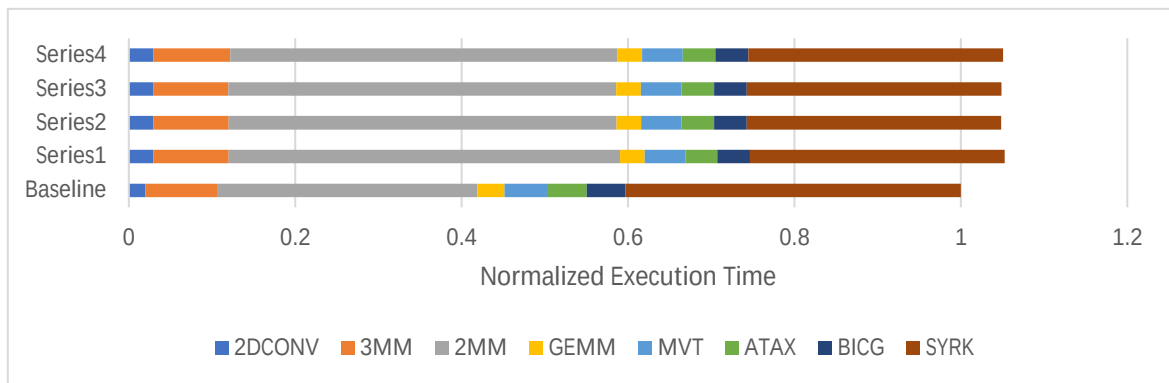


Figure 5-14. Normalized execution time in every series (10th execution)

In the 10th execution, all benchmarks perform nearly the same across all series and all series achieve more than 53% energy-saving with less than 5% performance loss, which proves our model has strong adaptability.

5.2.4 Experiment 4

In this experiment, we implement the action-decision step in RISC-V CPU developed by LPHPLAB NCKU [30, 31] for accurate circuit time. The modified architecture is presented in Figure 5-15. As Figure 5-16 illustrates, Host CPU transmits DDQN model and GPU features to RISC-V CPU. After RISC-V CPU completes calculation, Host CPU forwards action decision from RISC-V to GPU for frequency/voltage scaling.

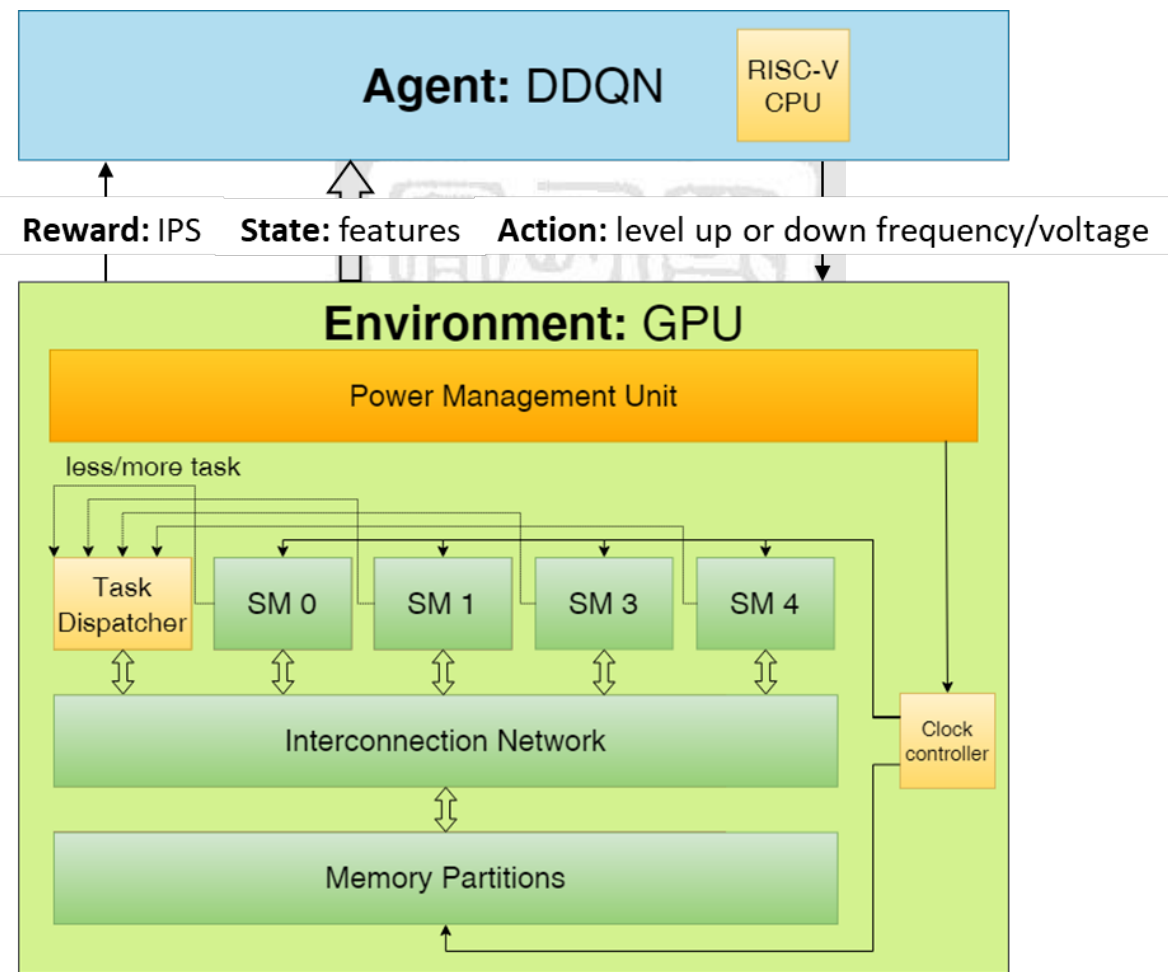


Figure 5-15. Sytem Architecture in Experiment 4

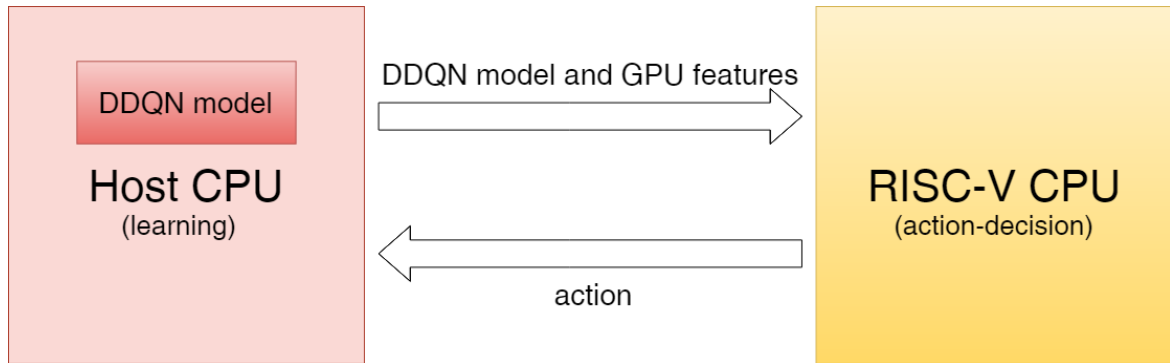


Figure 5-16. Action-decision on RISC-V CPU

Inference takes about $2.71\text{E}+06$ ns on RISC-V CPU. We divide $2.71\text{E}+06$ ns by 5 ns, which is the minimum period length of GPU, and take 542k cycles as the action-decision length. As for learning period, we estimate the length is twice as the inference length, because it takes one forward pass and one backward pass to complete the learning step. It takes $35\mu\text{S/V}$ and $38\mu\text{S/V}$ for voltage scaling up and down respectively referring to step response of 40nm DC/DC Converter [32]. Because every adjustment is 0.045V, the GPU will stop for 1575ns/1710ns when voltage scaling up/down.

Figure 5-17 shows the timeline of action-decision and learning flow. The GPU collects features for 5k cycles, and then sends them to DDQN agent. It takes about 542k cycles to decide action. The GPU will stop for 1575ns to level up frequency/voltage, and 1710ns to level down frequency/voltage. After this adjustment, the GPU continues execution until next sampling.

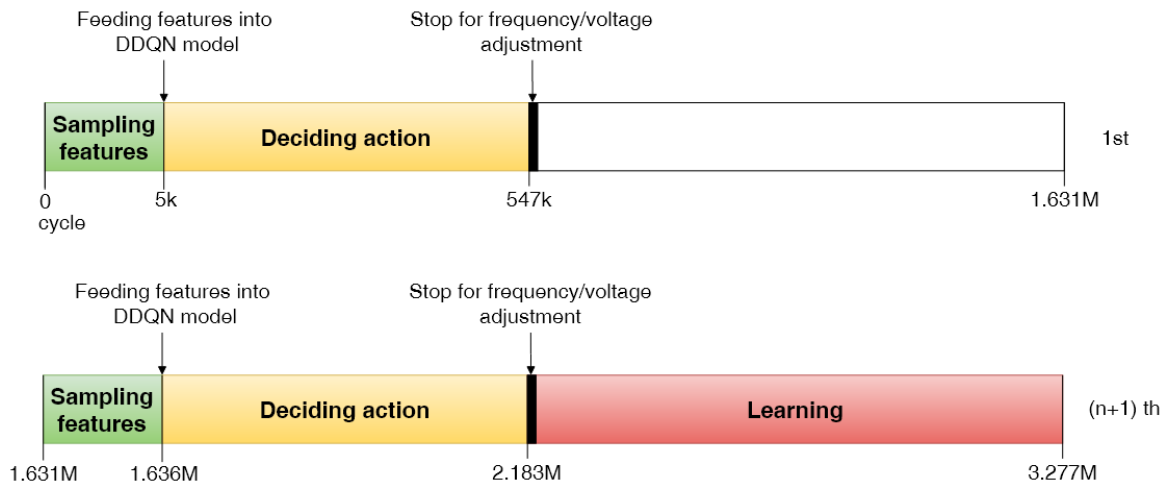


Figure 5-17. Timeline of action-decision and learning flow

Because the benchmarks length in Experiment 2 are too short for training sets collection, we enlarge these benchmarks to assure that enough training sets collected in one execution.

We train model which fits each benchmark by running each of them 5times after reset. We show energy and performance of the 5th execution in Figure 5-18 and Figure 5-19, respectively. The enlarge ratio is presented in parentheses. Take 2MM as an example. If the dimension of every matrix is $n \times n$ in Experiment 2, these matrices will be expanded to $2n \times 2n$ in this experiment.

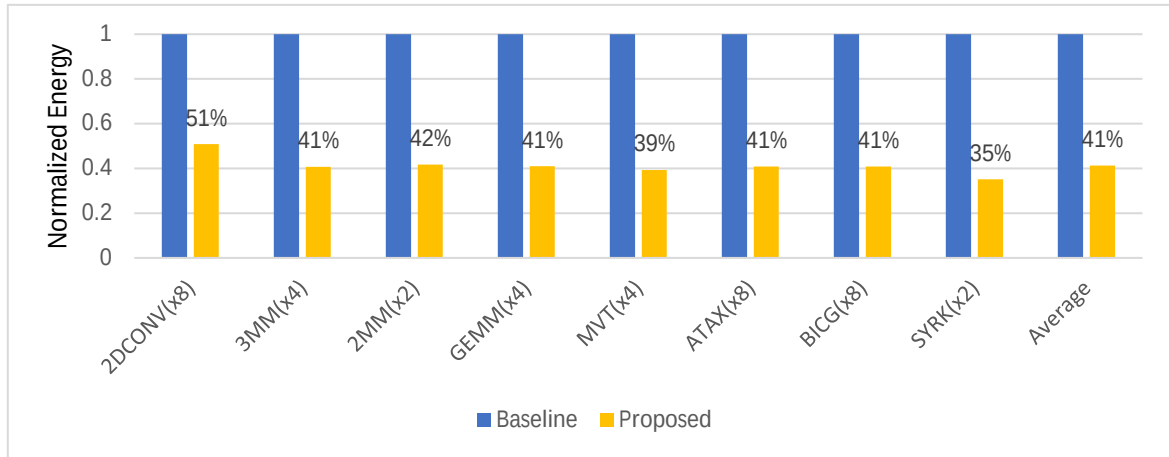


Figure 5-18. Energy when inference on RISC-V CPU

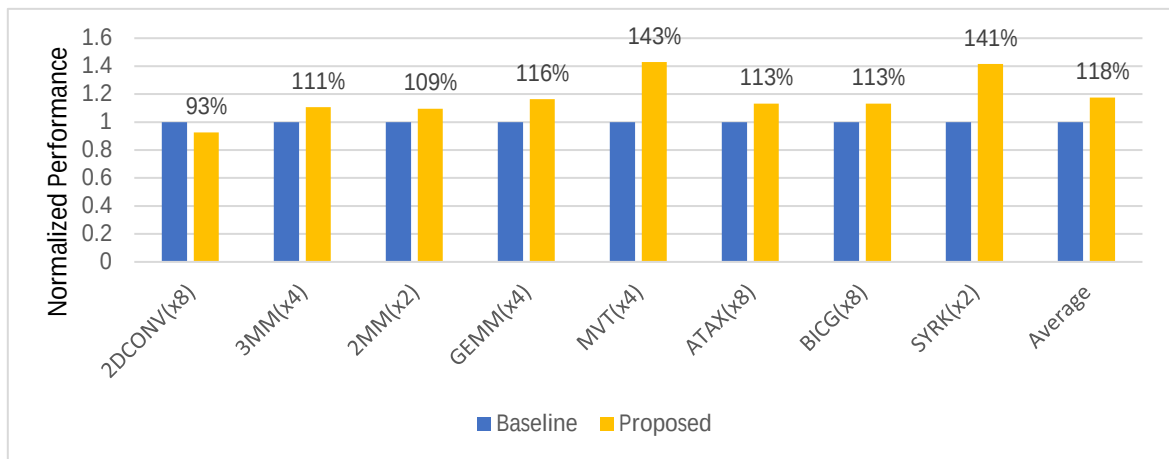


Figure 5-19. Performance when inference on RISC-V CPU

5.3 Overhead

We consider energy consumption caused by PLL, DC/DC converter, PMU, and RISC-V CPU here. As [33] and [32] states, PLL and DC/DC converter consumes about 2.8mW and 135 μ W in 40nm process, respectively. Action-decision takes about 0.155mJ on RISC-V CPU. We implement PMU algorithm by Verilog and synthesize it with 40nm process. The circuit cost 0.701mW.

We take execution of 3MM benchmark to calculate energy overhead. Since PLL, DC/DC converter, and RISC-V CPU only activate in action decision step; therefore, we multiply their energy by number of action-decision steps. In contrast, PMU keeps

collecting features from GPU; therefore, we multiply its power by GPU execution time.

The energy overhead is presented in Figure 5-20 and

Table 5-3 details contribution of every component. It takes at most 1710ns for voltage scaling, so the time overhead is 0.01% which is presented in Figure 5-21.

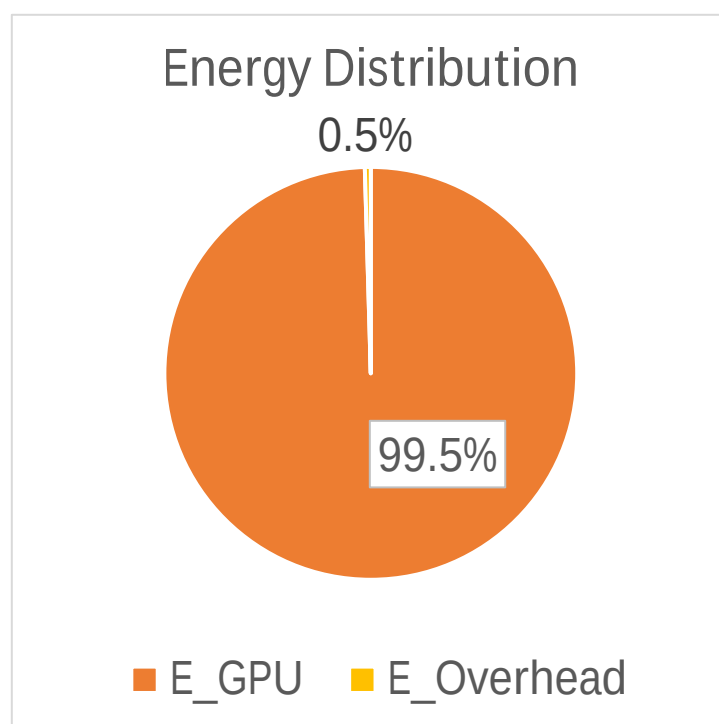


Figure 5-20. Energy overhead

Table 5-3. Detailed overhead of every component

Component	Energy consumption ratio
RISCV-CPU	0.42%
PLL	<0.01%
DC-DC converter	<0.01%
PMU	0.04%

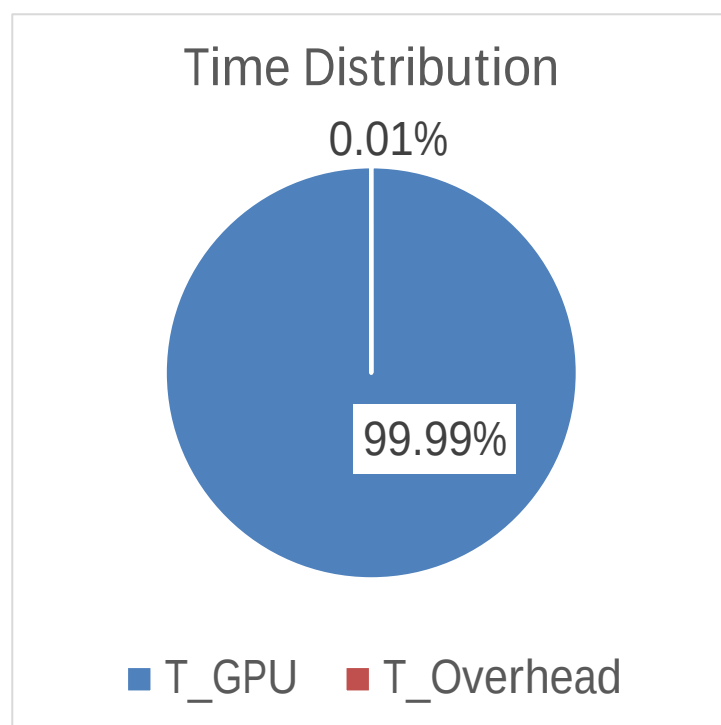


Figure 5-21. Time overhead

Chapter 6 Conclusions and Future Works

6.1 Conclusions

In this thesis, we propose a dynamic runtime system which tunes the number of tasks in every SM to reduce unnecessary power consumption in bottleneck conditions. Furthermore, we adopt DDQN to adjust core and memory frequency/voltage for energy-saving with less performance loss. For accurate inference time on hardware, we implement inference on RISC-V CPU.

Across 8 kernels, the mechanism achieves 59% energy savings with 18% performance improvement on average. Moreover, the model shows strong adaptability when running programs which is interleaved by compute-intensive and memory-intensive benchmarks. The energy-saving is more than 53% with less than 5% performance loss no matter in which sequences.

6.2 Future Works

RISC-V CPU takes 2.71 ms to predict action which is much longer than the sampling time. Therefore, it is suggested to add MAC instruction and floating operations in RISC-V CPU for faster calculation and better accuracy for learning on RISC-V CPU.

Reference

- [1] A. Coates, B. Huval, T. Wang, D. Wu, B. Catanzaro, and N. Andrew, "Deep Learning with COTS HPC Systems," in *Proc. International Conference on Machine Learning*, 2013, pp. 1337-1345.
- [2] R. Raina, A. Madhavan, and A. Y. Ng, "Large-Scale Deep Unsupervised Learning Using Graphics Processors," in *Proc. 26th Annual International Conference on Machine Learning*, 2009, pp. 873-880.
- [3] S. Shi, Q. Wang, P. Xu, and X. Chu, "Benchmarking state-of-the-art deep learning software tools," in *Proc. International Conference on Cloud Computing and Big Data (CCBD)*, 2016, pp. 99-104.
- [4] M. McNaughton, C. Urmson, J. M. Dolan, and J.-W. Lee, "Motion Planning for Autonomous Driving with a Conformal Spatiotemporal Lattice," in *Proc. IEEE International Conference on Robotics and Automation*, 2011, pp. 4889-4895.
- [5] Q. Zhang, M. Lin, L. T. Yang, Z. Chen, and P. Li, "Energy-Efficient Scheduling for Real-Time Systems Based on Deep Q-Learning Model," *IEEE Trans. on Sustainable Computing*, vol. 4, no. 1, pp. 132-141, 2019.
- [6] S. F. Region. "Edge Computing Market Size, Share & Trends Analysis Report By Component."
<https://www.grandviewresearch.com/industry-analysis/edge-computing-market>
(accessed Oct. 12, 2020).
- [7] M. Gebhart *et al.*, "Energy-Efficient Mechanisms for Managing Thread Context in Throughput Processors," in *Proc. 38th Annual International Symposium on Computer Architecture (ISCA)*, 2011, pp. 235-246.
- [8] M. Gebhart, S. W. Keckler, B. Khailany, R. Krashinsky, and W. J. Dally, "Unifying Primary Cache, Scratch, and Register File Memories in a Throughput Processor," in *Proc. 45th Annual IEEE/ACM International Symposium on Microarchitecture*, 2012, pp. 96-106.
- [9] S. Hong and H. Kim, "An Integrated GPU Power and Performance Model," in *Proc. 37th Annual International Symposium on Computer Architecture*, Saint-Malo, France, 2010, pp. 280-289.
- [10] Y. Wang and N. Ranganathan, "An Instruction-Level Energy Estimation and Optimization Methodology for GPU," in *Proc. IEEE 11th International Conference on Computer and Information Technology*, 2011, pp. 621-628.
- [11] G. Wang, Y. Lin, and W. Yi, "Kernel Fusion: an Effective Method for Better Power Efficiency on Multithreaded GPU," in *Proc. IEEE/ACM Int'l Conference*

- on *Green Computing and Communications & Int'l Conference on Cyber, Physical and Social Computing*, 2010, pp. 344–350.
- [12] Y. Yang, P. Xiang, M. Mantor, and H. Zhou, "Fixing Performance Bugs: an Empirical Study of Open-Source GPGPU Programs," in *Proc. 41th International Conference on Parallel Processing*, 2012, pp. 329-339.
 - [13] Y. Jiao, H. Lin, P. Balaji, and W. Feng, "Power and Performance Characterization of Computational Kernels on the GPU," in *Proc. IEEE/ACM Int'l Conference on Green Computing and Communications & Int'l Conference on Cyber, Physical and Social Computing*, 2010, pp. 221-228.
 - [14] K. Ma, X. Li, W. Chen, C. Zhang, and X. Wang, "GreenGPU: A Holistic Approach to Energy Efficiency in GPU-CPU Heterogeneous Architectures," in *Proc. International Conference on Parallel Processing*, 2012, pp. 48-57.
 - [15] X. Mei, L. S. Yung, K. Zhao, and X. Chu, "A Measurement Study of GPU DVFS on Energy Conservation," in *Proc. Workshop on Power-Aware Computing and Systems*, Farmington, Pennsylvania, 2013, pp. 89-100.
 - [16] A. Sethia and S. Mahlke, "Equalizer: Dynamic Tuning of GPU Resources for Efficient Execution," in *Proc. 47th Annual IEEE/ACM International Symposium on Microarchitecture*, 2014, pp. 647-658.
 - [17] Q. Jiao, M. Lu, H. P. Huynh, and T. Mitra, "Improving GPGPU Energy-Efficiency Through Concurrent Kernel Execution and DVFS," in *Proc. IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2015, pp. 1-11.
 - [18] J. Li, B. Guo, Y. Shen, D. Li, and Y. Huang, "A Modeling Approach for Energy Saving Based on GA-BP Neural Network," *Journal of Electrical Engineering and Technology*, vol. 11, pp. 1289-1298, Sep. 2016.
 - [19] A. Majumdar, L. Piga, I. Paul, J. L. Greathouse, W. Huang, and D. H. Albonesi, "Dynamic GPGPU Power Management Using Adaptive Model Predictive Control," in *Proc. IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2017, pp. 613-624.
 - [20] K. Fan, B. Cosenza, and B. Juurlink, "Predictable GPUs Frequency Scaling for Energy and Performance," in *Proc. 48th International Conference on Parallel Processing*, Kyoto, Japan, 2019, pp. 1-10.
 - [21] J. Guerreiro, A. Ilic, N. Roma, and P. Tomás, "DVFS-aware Application Classification to Improve GPGPUs Energy Efficiency," *Parallel Computing*, vol. 83, pp. 93-117, Apr. 2019.
 - [22] S. Dey, A. Singh, X. Wang, and K. McDonald-Maier, "User Interaction Aware Reinforcement Learning for Power and Thermal Efficiency of CPU-GPU Mobile

- MPSoCs," in *Proc. Design, Automation, and Test in Europe 2020 (DATE 2020)*, Grenoble, France, 2020, pp. 1728-1733.
- [23] A. Choudhary. "A Hands-On Introduction to Deep Q-Learning using OpenAI Gym in Python." <https://www.analyticsvidhya.com/blog/2019/04/introduction-deep-q-learning-python/> (accessed 2020, Aug. 28).
- [24] A. Oppermann. "Self Learning AI-Agents III:Deep (Double) Q-Learning." <https://towardsdatascience.com/deep-double-q-learning-7fca410b193a> (accessed 2020, Aug. 28).
- [25] J.-T. Syu, "Intelligent Power Management Based on Reinforcement Learning for General-Purpose Computing on Graphics Processing Units," Master's Thesis, Department of Electrical Engineering, National Cheng Kung University, Tainan, Taiwan, 2019.
- [26] W.-S. Hsieh, "Micro-Architecture Optimization of HSA-Compatible GPU," Master's Thesis, Department of Electrical Engineering, National Cheng Kung University, Tainan, Taiwan, 2016.
- [27] J.-H. Jheng, "Design of Cycle-accurate SIMT Core and Implementation," Master's Thesis, Department of Electrical Engineering, National Cheng Kung University, Tainan, Taiwan, 2018.
- [28] *DDR4 SDRAM*, J. S. S. T. Association, 2012. [Online]. Available: http://www.softnology.biz/pdf/JESD79-4_DDR4_SDRAM.pdf?fbclid=IwAR1doOEVRs8Y6UK3JsNjaiUs9gvu1OhXSxNVUjVbKFAc8kxROPRAAN8T58A
- [29] C.-H. Chang, "An Intelligent Thermal Control Method for General-Purpose Graphic Processing Units," Master's Thesis, Department of Electrical Engineering, National Cheng Kung University, Tainan, Taiwan, 2020.
- [30] C.-H. Chiang, "Design Exploration of a Lightweight Hardware Secure Microsystem using Physical Unclonable Function at Electronic System Level," Master's Thesis, Department of Electrical Engineering, National Cheng Kung University, Tainan, Taiwan, 2018.
- [31] C.-W. Chang, "A Nonvolatile Processor Virtual Platform for Energy Efficiency and Performance Exploration," Master's Thesis, Department of Electrical Engineering, National Cheng Kung University, Tainan, Taiwan, 2018.
- [32] J. Shi, Y.-C. Hsu, E. Soenen, A. Roth, and J. Gaither, "A Wide-Range DC/DC Converter with 2nd Order Digital Compensation and Direct Battery Connection in 40nm CMOS," in *Proc. Custom Integrated Circuits Conference*, 2011, pp. 1-4.

- [33] Y. Hengzhou, G. Yang, and M. Zhuo, "A 40nm/65nm Process Adaptive Low Jitter Phase-Locked Loop," in *Proc. International Symposium on Integrated Circuits (ISIC)*, 2014, pp. 500-503.

